

CS 182 — Deep Neural Networks

Fall 2025

Contents

Lecture 10: Finish Normalization, Dropout and start ResNets	6
10.1 Convolutional Neural Networks (Continued)	6
10.1.1 Two Perspectives on a $k \times k$ Convolution	6
10.2 Training: Stability and Effectiveness	7
10.2.1 Normalization Layers	7
10.2.2 Dropout: In-Network Data Augmentation	9
10.3 Residual and Skip Connections	10
10.3.1 The Problem with Deep Networks	10
10.3.2 The Solution: Skip Connections (ResNets)	10
Lecture 11: Finish ResNets, fully convolutional nets and U-nets	12
11.1 Residual and Skip Connections (Recap)	12
11.1.1 The Degradation Problem	12
11.1.2 The Gradient Superhighway	12
11.2 Inside a Residual Block	13
11.2.1 Core Structure and Intuition	13
11.2.2 Placement of Normalization Layers	13
11.2.3 Handling Dimension Mismatches	14
11.3 A Modern Perspective: ConvNeXt	14
11.4 Upsampling and Transpose Convolutions	15
11.4.1 Upsampling Methods	15
11.4.2 Transpose Convolution	16
11.5 Architectures for Image-Level Tasks: U-Nets	16
11.5.1 The Bottleneck Problem	16
11.5.2 The U-Net Solution	16
Lecture 12: Graph Neural Nets	18
12.1 Introduction and Resources	18

12.2	Recap and A Loose End from CNNs	18
12.3	Generalizing CNNs to Graphs	19
12.4	A Conceptual Bridge: Viewing an Image as a Graph	19
12.5	Handling General Graph Structures	21
12.6	The Attention Mechanism in GNNs	22
Lecture 13: DiffPool and finish GNNs, Start RNNs		24
13.1	Administrative Notes	24
13.1.1	Reading Material	24
13.1.2	Project Information	24
13.2	Finishing Graph Neural Networks (GNNs)	24
13.2.1	The Challenge of Pooling and Downsampling in GNNs	24
13.2.2	A Principled Approach: Differentiable Pooling (DiffPool)	25
13.3	Final GNN Challenge: Handling Held-Out Data	28
13.3.1	Approach 1: Excise Entire Nodes	28
13.3.2	Approach 2: Mask Node Content	28
13.4	Introduction to Recurrent Neural Networks (RNNs)	28
13.4.1	Historical Context	28
13.4.2	The Core Idea: Sequential and Causal Data	29
13.4.3	A Signal-Processing Perspective	29
Lecture 14: RNNs and Self-Supervision		30
14.1	Introduction to Recurrent Neural Networks (RNNs)	30
14.1.1	Context and History	30
14.1.2	A Signal-Processing Perspective	30
14.2	Designing Modern RNNs	31
14.2.1	Generalizing from CNN Principles	31
14.2.2	Traditional "Vanilla" RNNs	31
14.3	Training RNNs: Backpropagation Through Time (BPTT)	32
14.3.1	Gradient Flow	32
14.3.2	The Problem of Long-Range Dependencies	32
14.3.3	A Modern View: Parallelism during Training	32

14.4	Example: Learning a Kalman Filter	33
14.4.1	Problem Setup	33
14.4.2	Case 1: Supervised Learning	33
14.4.3	Case 2: The Need for Self-Supervision	34
14.5	Self-Supervision	34
14.5.1	General Principle	34
14.5.2	Self-Supervised Learning for the Filter Problem	34
14.5.3	Key Lessons from the Example	35
14.6	Connection to Unsupervised Learning	35
14.6.1	Thinking about PCA as Self-Supervision	35
Lecture 15: Finish Self-Supervision and Start State-Space Models		36
15.1	Self-Supervision	36
15.1.1	General Principles	36
15.1.2	Connecting to Classical Unsupervised Learning	36
15.1.3	Reinterpreting PCA: The Autoencoder	37
15.1.4	Variations of Autoencoders	38
15.1.5	Contrastive Self-Supervision	39
15.2	State-Space Models (SSMs)	40
15.2.1	Motivation: Problems with Traditional RNNs	40
15.2.2	The Key Idea: Eliminate Horizontal Non-Linearity	40
15.2.3	The Convolution View for Parallel Training	40
15.2.4	Improving Computational Efficiency	41
15.2.5	Expressive Power and Inference	42
Lecture 16: Finish State-Space Models		43
16.1	Recap: State-Space Models (SSMs) for Sequence Modeling	43
16.1.1	The SSM Approach: Linear Time-Invariant (LTI) Systems	43
16.2	Efficient Parallel Training for SSMs (The S4 Model)	44
16.2.1	Challenge 1: Computing Powers of Matrix A	44
16.2.2	Challenge 2: Applying FFT to Vector Sequences	44
16.2.3	Challenge 3: Reducing the Number of Convolutions	44

16.2.4	Challenge 4: Capturing Long-Range Dependencies	45
16.3	Mamba: Selective State-Space Models (S6)	46
16.3.1	A New Perspective: Continuous-Time Systems	46
16.3.2	Learning to be Selective with Δ	46
16.3.3	Implications for Training	47
16.3.4	The Mamba Architecture Block	47
Lecture 17: Attention and Transformers		48
17.1	Introduction: From State-Space Models to Attention	48
17.2	Sequence-to-Sequence Problems with RNNs	48
17.2.1	The Encoder-Decoder Architecture	48
17.2.2	Visualizing the Architecture and the Bottleneck	48
17.3	Training Sequence-to-Sequence Models	49
17.3.1	Teacher Forcing	49
17.3.2	Autoregressive Generation (Inference Time)	49
17.4	Overcoming the Bottleneck: The Motivation for Attention	49
17.4.1	Analogy to U-Nets	50
17.4.2	The Challenge in Language	50
17.5	Developing the Attention Mechanism: A Step-by-Step Intuition	50
17.5.1	Idea -1: Exact Match	50
17.5.2	Idea 0: Closest Match	51
17.5.3	Idea 1: Weighted Average	51
17.6	The Attention Formula and its Components	51
17.6.1	Making it Learnable: Q, K, V Projections	51
17.6.2	Scaled Dot-Product Attention	52
17.6.3	Types of Attention	52
17.7	Multi-Head Attention	52
Lecture 18: Transformers		54
18.1	Recap: Sequence-to-Sequence (Seq2Seq) Models	54
18.1.1	Encoder-Decoder Architecture	54
18.1.2	The Bottleneck Problem	54

18.1.3	Training Seq2Seq Models	54
18.2	The Attention Mechanism	54
18.2.1	Conceptual Overview: Queries, Keys, and Values	55
18.2.2	Calculating Q, K, and V	55
18.2.3	Types of Attention	55
18.3	Scaled Dot-Product Attention	56
18.3.1	The Attention Formula	56
18.3.2	The Need for Scaling	56
18.4	Multi-Head Attention	56
18.4.1	Mechanism	57
18.4.2	Variations for Efficiency	57
18.5	The Transformer Architecture	57
18.5.1	Overall Structure	57
18.5.2	The Encoder Block	57
18.5.3	The Decoder Block	58
18.6	Embedding and Positional Encoding	58
18.6.1	Input/Output Embeddings	58
18.6.2	Positional Encoding	59
Lecture 19: Finish Transformers		60
19.1	Recap and Introduction	60
19.2	Rotary Positional Encoding (RoPE)	60
19.2.1	The Core Idea of RoPE	60
19.2.2	RoPE in Two Dimensions	61
19.2.3	Extending RoPE to Higher Dimensions	61
19.3	Key Transformer Architectures	62
19.3.1	GPT-3: The Decoder-Only Model	62
19.3.2	BERT: The Encoder-Only Model	63
19.4	Modern Architectural Components	64
19.4.1	Gated Linear Units (GLU)	64
19.4.2	Mixture of Experts (MoE)	64
19.5	Putting It All Together: A Modern Architecture	65

Lecture 10: Finish Normalization, Dropout and start ResNets

10.1 Convolutional Neural Networks (Continued)

We continue our discussion of Convolutional Neural Networks (CNNs). The core idea remains using a convolutional architecture for weight sharing, which is particularly effective for image processing. We will first clarify a key concept from the previous lecture: the two perspectives on a $k \times k$ convolution.

10.1.1 Two Perspectives on a $k \times k$ Convolution

Consider a convolutional layer with a $k \times k$ filter, C_{in} input channels, and C_{out} output channels. The number of parameters is identical regardless of perspective: $k^2 \cdot C_{\text{in}} \cdot C_{\text{out}}$ weights and C_{out} biases.

Perspective 1: C_{out} Separate Filters This perspective views the operation as applying C_{out} distinct $k \times k$ convolutional filters. Each of these filters has:

- C_{in} input channels.
- 1 output channel.

The process for a single output channel is as follows:

1. A single $k \times k \times C_{\text{in}}$ filter is applied to a patch of the input tensor.
2. For each of the C_{in} input channels, there is a corresponding $k \times k$ matrix of real-valued weights within the filter.
3. The values in the input patch are multiplied element-wise with the corresponding filter weights, and all results are summed together.
4. A single bias term is added to this sum to produce one value in the output feature map.

This entire process is repeated C_{out} times with C_{out} different filters to generate the C_{out} output channels.

Perspective 2: A Single Filter with Weight Matrices This perspective views the operation as a single $k \times k$ convolution where the components are not scalars, but matrices and vectors.

- Each of the k^2 spatial positions in the filter contains a **weight matrix** of shape $C_{\text{in}} \times C_{\text{out}}$.
- There is a single **bias vector** $\vec{b}$ of dimension C_{out} for the entire convolution.

This perspective treats the values at each pixel as a vector. For an RGB image, this is a 3-dimensional vector. Deeper in the network, it is a C_{in} -dimensional vector. The convolution operation at a single output pixel is then a linear transformation on vectors:

$$\vec{h}_{\text{out}} = \vec{b} + \sum_{i \in \text{neighborhood}} W_i \cdot \vec{h}_{\text{in},i}$$

Where:

- $\vec{h}_{\text{out}}$ is the C_{out} -dimensional output vector at a pixel.
- $\vec{b}$ is the C_{out} -dimensional bias vector.
- The summation index i ranges over the k^2 spatial positions in the convolutional filter's neighborhood (e.g., for a 3×3 filter, this includes center, north, south, east, west, etc.).
- W_i is the $C_{\text{out}} \times C_{\text{in}}$ weight matrix for the i -th spatial position in the filter.
- $\vec{h}_{\text{in},i}$ is the C_{in} -dimensional input vector from the corresponding spatial position in the input feature map.

Both perspectives are computationally identical and represent the same affine transformation from one image representation to another.

10.2 Training: Stability and Effectiveness

To train deep networks effectively, we must manage the scale of activations and gradients to prevent them from exploding or vanishing. This leads to the idea of inserting normalization layers within the network architecture.

10.2.1 Normalization Layers

The core idea is inspired by input standardization. We want to control the distribution of activations at hidden layers.

RMS Norm Layer A simple approach is to normalize the activations to have a unit Root Mean Square (RMS) norm. For a d -dimensional input vector $\vec{h}$, the normalized output $\tilde{\vec{h}}$ is:

$$\tilde{\vec{h}} = \frac{\vec{h}}{\|\vec{h}\|_{\text{rms}}} \quad \text{where} \quad \|\vec{h}\|_{\text{rms}} = \sqrt{\frac{1}{d} \sum_{i=1}^d h_i^2 + \epsilon}$$

The small constant $\epsilon > 0$ is added for numerical stability to prevent division by zero.

Batch Normalization (Batch Norm) Batch norm standardizes the inputs for each feature channel separately, using statistics computed over the current mini-batch.

- **Averaging Domain:** For a given channel, statistics are computed by averaging over all spatial dimensions (height H , width W) and all examples in the mini-batch (size N).
- Let $\mathcal{B}$ be the set of activations for a single channel across the batch and spatial dimensions. The size of $\mathcal{B}$ is $N \times H \times W$.
- The mean m and standard deviation s are computed as:

$$m = \frac{1}{|\mathcal{B}|} \sum_{h_i \in \mathcal{B}} h_i$$

$$s = \sqrt{\frac{1}{|\mathcal{B}|} \sum_{h_i \in \mathcal{B}} (h_i - m)^2 + \epsilon}$$

- The normalization is then applied element-wise:

$$\hat{h}_i = \frac{h_i - m}{s}$$

- **Learnable Parameters:** Batch norm introduces learnable per-channel scale (γ) and shift (δ) parameters. This allows the network to learn the optimal mean and variance for the activations, rather than being fixed at 0 and 1.

$$h_{\text{out}} = \gamma \hat{h}_i + \delta$$

These are initialized with $\gamma = 1$ and $\delta = 0$. Typically, weight decay is not applied to these parameters.

Layer Normalization (Layer Norm) Layer norm computes statistics differently, making it independent of the batch size.

- **Averaging Domain:** For a single example in the batch, statistics are computed by averaging over all channels (C) and all spatial dimensions (H, W).
- The mean and standard deviation are computed for each example in the batch independently.
- This approach works well even with a batch size of 1.
- Similar to batch norm, layer norm can also have learnable per-channel scale (γ) and shift (δ) parameters.

Other Variants

- **Instance Normalization:** Statistics are computed per-channel and per-example, averaging only over the spatial dimensions (H, W).

- **RMS Norm Variants:** Any of these normalization schemes can be implemented without the recentring step (i.e., without computing and subtracting the mean m). This is often referred to as RMSNorm and is typically done in the style of Layer Norm.

Normalization at Test/Inference Time A key issue arises with methods that depend on batch statistics.

- **Layer Norm:** No issue. Since it operates on a single example at a time, the procedure at test time is identical to training.
- **Batch Norm:** At inference time, there is no batch to compute statistics over. We need a fixed mean m and standard deviation s . Several strategies exist:
 1. **Use Training Statistics:** During training, maintain a running exponential moving average of the mean and variance for each channel. These moving averages are then "frozen" and used at inference time. A common practice is to compute these final statistics over the entire training set after training is complete (e.g., by running an extra epoch with no gradient updates).
 2. **Test-Time Adaptation:** For applications where the data distribution might shift (e.g., autonomous driving), one can continue to update the running average of m and s during inference.
 3. **Learn as Hyperparameters:** Treat m and s as parameters to be learned on a held-out validation set.

10.2.2 Dropout: In-Network Data Augmentation

Just as normalization is an in-network analogue to input standardization, dropout can be seen as an in-network analogue to data augmentation.

- **Idea:** During training, randomly set a fraction of neuron activations to zero at each update step.
- **Purpose:** This prevents neurons from co-adapting too much and forces the network to learn more robust features. It encourages the internal representations to be **redundant**, meaning the necessary information is encoded in multiple ways, so that dropping some activations does not destroy the information.
- **Mechanics:** Typically used for MLP-style layers (including 1×1 convolutions).
 - For each element in the activation vector $\vec{h}_L$, it is kept with a probability p (the "keep probability") and set to zero with probability $1 - p$.
 - This is equivalent to an element-wise multiplication of the activation vector with a mask vector of i.i.d. Bernoulli random variables.
- **Dropout at Test Time:**

- **Problem:** During training, the expected output of a layer is scaled down by the keep probability p . If we do nothing at test time (i.e., keep all activations), the scale of the activations will be significantly larger than what the network was trained on. This mismatch in scale disrupts the network's performance.
- **Solution (Inverted Dropout):** To compensate for the scale change, the standard approach is to perform the scaling during training. This is known as inverted dropout.

$$\tilde{h}_L[j] = \begin{cases} 0 & \text{with probability } 1 - p \\ \frac{h_L[j]}{p} & \text{with probability } p \end{cases}$$

By scaling up the activations that are kept during training, their expected value remains unchanged: $E[\tilde{h}_L[j]] = p \cdot \frac{h_L[j]}{p} + (1 - p) \cdot 0 = h_L[j]$.

- With inverted dropout, no modification is needed at test time; the dropout layer simply acts as an identity function. This is the standard implementation in frameworks like PyTorch.

10.3 Residual and Skip Connections

As networks get deeper, a "degradation" problem can occur: adding more layers can lead to higher training error, not just higher test error. This suggests an optimization problem, not an overfitting one. A 56-layer network might perform worse on both training and test sets than a 20-layer network.

10.3.1 The Problem with Deep Networks

Two main hypotheses explain this degradation:

1. **Vanishing/Exploding Gradients:** During backpropagation, gradients are repeatedly multiplied by weight matrices. In deep networks, this can cause the gradient signal to shrink exponentially to zero (vanish) or grow exponentially to infinity (explode), making it difficult for early layers to learn.
2. **Corrupted Gradient Signals ("Confused Gradients"):** In a randomly initialized deep network, layers are not well-configured. The gradient signal passed from a later layer to an earlier one can be noisy and uninformative. This is like a game of telephone where the useful error signal from the loss function gets corrupted as it propagates backward through many layers.

10.3.2 The Solution: Skip Connections (ResNets)

The core idea is to create "gradient superhighways" that allow the gradient to flow more directly to earlier layers.

- A naive approach might be to connect the output of every earlier layer to the input of every

later layer (as in DenseNets). However, this can lead to a blow-up in the number of channels and activations.

- A more minimalist and highly effective approach is the **skip connection** or **residual connection**.
- A layer or a block of layers is reformulated to learn a *residual function* $F(x)$ with respect to its input x . The output of the block becomes:

$$H(x) = F(x) + x$$

The "+ x " part is the skip connection, which is an identity mapping that bypasses the layers.

- **Benefit for Gradient Flow:** This architecture creates a direct path for the gradient. Consider the derivative of the loss $\mathcal{L}$ with respect to the input x :

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial H(x)} \frac{\partial H(x)}{\partial x} = \frac{\partial \mathcal{L}}{\partial H(x)} \left(\frac{\partial F(x)}{\partial x} + 1 \right)$$

The "+1" term ensures that the gradient can flow backward through the identity path unimpeded, even if the gradients through the weight layers $\frac{\partial F(x)}{\partial x}$ become very small. This effectively combats the vanishing gradient problem and allows for the successful training of much deeper networks.

Lecture 11: Finish ResNets, fully convolutional nets and U-nets

11.1 Residual and Skip Connections (Recap)

11.1.1 The Degradation Problem

As neural networks became more successful, researchers observed that simply making them deeper (adding more layers) did not always lead to better performance. In fact, a counter-intuitive phenomenon called the **degradation problem** was discovered: a deeper network (e.g., 56 layers) would perform worse than a shallower one (e.g., 20 layers) on *both* the training and testing sets.

This is puzzling from a machine learning perspective. A deeper model has more parameters and should, in theory, be able to at least learn the identity function for the extra layers, thereby performing no worse than its shallower counterpart. The poor performance on the training set suggests an optimization issue, not overfitting. The hypothesis was that as gradients propagated backward through many layers, they would either diminish (vanish) or become distorted and less informative, making it difficult for the earlier layers to learn effectively.

11.1.2 The Gradient Superhighway

The solution proposed was to create a more direct path for gradients to flow from later layers back to earlier ones. This concept is often called a **gradient superhighway**. Since gradients in backpropagation follow the forward paths of the network in reverse, creating a gradient highway requires adding new paths in the forward pass.

The idea is to add **skip connections** (or residual connections), which take the input to a block of layers, x , and add it to the output of that block, $F(x)$. The new output is then $H(x) = F(x) + x$.

- A **block** is a more abstract concept than a single layer. It can contain multiple layers, such as convolutions, non-linearities, and normalizations.
- The addition operation acts as a gradient distributor. During backpropagation, the gradient flowing into the sum node is passed identically to both the block's output and the input skip path.
- This creates direct gradient paths from the final loss all the way back to the initial layers, bypassing many intermediate blocks. This ensures that even the earliest layers receive clean, informative gradients.

A common initial thought was to connect every layer to every subsequent layer. However, this causes the activations to grow excessively large. Skip connections provide a minimal and effective way to achieve the goal of unimpeded gradient flow. The activations grow arithmetically with depth (due to the summation), which is manageable with standard floating-point precision like FP32, but can become a concern with lower-precision formats (FP16, FP8, FP4).

11.2 Inside a Residual Block

11.2.1 Core Structure and Intuition

The minimal and effective structure for a block, ‘ $F(x)$ ’, that is skipped over consists of at least two weight layers. A typical core structure is:

$$\text{Input} \rightarrow [\text{Weight Layer 1}] \rightarrow [\text{Bias}] \rightarrow [\text{ReLU}] \rightarrow [\text{Weight Layer 2}] \rightarrow [\text{Bias}] \rightarrow \text{Output}$$

The reason two layers are needed is rooted in the expressive power of the block. A single layer followed by a ReLU would restrict the block’s output ‘ $F(x)$ ’ to be non-negative. This is a significant limitation. The second weight layer allows the block to produce both positive and negative values, enabling it to learn more complex functions.

An ODE Perspective The structure of a ResNet changes the fundamental composition of functions in a deep network.

- **Pre-ResNet (Nested Functions):** $y = f_n(f_{n-1}(\dots f_1(x)\dots))$
- **ResNet (Additive Updates):** $x_{i+1} = x_i + g_i(x_i)$

This additive form is reminiscent of a forward Euler discretization of an Ordinary Differential Equation (ODE), such as $\frac{dx}{dt} = g(x(t), t)$. This analogy inspired the field of **Neural ODEs**, where the network is viewed as learning a continuous transformation (a vector field) that flows an input to its desired output. From this perspective, it’s clear why $g_i(x_i)$ must be able to produce negative values—to allow the “flow” to move in any direction, not just forward.

This perspective also inspired the idea of **weight sharing across depth**, where the same block function g is used repeatedly, analogous to a time-invariant ODE. While this is effective for many physics-inspired problems, standard ResNets with unique weights at each block generally perform better on computer vision tasks.

11.2.2 Placement of Normalization Layers

A key design choice is where to place normalization layers (like Batch Normalization or Layer Normalization) in relation to the residual block.

- **Old Perspective (Post-Normalization):** Early ResNet papers and related architectures (e.g., the original Transformer) often placed normalization layers at the end of a block, after the addition with the skip connection. The logic was to re-normalize the activations after they might have grown or shifted due to the addition.
- **Modern Default (Pre-Normalization):** The current standard practice is to place the normalization layer *inside* the block, at the very *beginning*.

A modern pre-norm residual block looks like this:

Input $x_i \rightarrow [\text{Normalization}] \rightarrow [\text{Weight Layer 1}] \rightarrow [\text{ReLU}] \rightarrow [\text{Weight Layer 2}] \rightarrow \text{Output } F(x_i)$

The final output of the entire stage is then $x_{i+1} = x_i + F(x_i)$.

The key advantage of the pre-norm convention is that there are **no operations on the skip path**. The "gradient superhighway" is a clean, uninterrupted path of additions from the end of the network to the beginning. This leads to more stable training, especially for very deep networks.

11.2.3 Handling Dimension Mismatches

Standard skip connections $H(x) = F(x) + x$ require that the input x and the output of the block $F(x)$ have the same shape. This is often not the case, especially at stages where downsampling occurs (e.g., using a strided convolution or a pooling layer inside the block), which reduces spatial dimensions and typically increases the number of channels.

If an input is $100 \times 100 \times 32$ and the block output is $50 \times 50 \times 64$, a simple addition is impossible. To maintain the gradient highway, we must modify the skip connection path to match the dimensions.

- **Goal:** Create a path for gradients to flow back, even if shapes differ.
- **Solution:** Introduce a projection layer, P , on the skip path. The new formulation becomes $H(x) = F(x) + P(x)$.
- **Choices for P:**
 1. **Learnable Projection:** The most flexible approach is to make P a learnable linear layer (e.g., a 1×1 convolution with the appropriate stride and output channels). The network learns the best way to project the identity.
 2. **Fixed Projection:** A simpler method is to use a fixed function. For example, downsample spatially via averaging or subsampling, and handle channel changes by padding with zeros or truncating.

The crucial point is that *some* path must be created. Simply omitting the skip connection at these transition points would break the gradient superhighway.

11.3 A Modern Perspective: ConvNeXt

In 2022, a paper titled "A ConvNet for the 2020s" revisited the ResNet architecture, modernizing it with design choices inspired by Transformers, resulting in an architecture called **ConvNeXt**. This work demonstrated that with the right details, pure ConvNets could still outperform Transformer-based models on vision tasks.

The key changes from a standard ResNet block to a ConvNeXt block are:

1. Inverted Bottleneck:

- **ResNet:** Uses a "bottleneck" design: 1×1 conv (reduce channels) $\rightarrow 3 \times 3$ conv $\rightarrow 1 \times 1$ conv (restore channels).

- **ConvNeXt:** Uses an "inverted bottleneck": 1×1 conv (expand channels) $\rightarrow k \times k$ conv $\rightarrow 1 \times 1$ conv (reduce channels). This allows the network to operate on a higher-dimensional feature space internally, increasing expressive power.

2. Depthwise Separable Convolutions:

- The main spatial convolution (e.g., 7×7) is replaced with a **depthwise convolution**.
- In a depthwise convolution, each input channel is processed independently by its own 2D filter. There is no mixing of information across channels.
- The channel mixing is handled entirely by the preceding and succeeding 1×1 convolutions (also known as pointwise convolutions).
- This is far more parameter-efficient than a standard convolution.

3. Fewer Normalization and Activation Layers:

- Instead of placing normalization (BN) and activation (ReLU) layers after each convolution, ConvNeXt uses a single **Layer Normalization (LN)** at the beginning of the block.
- It uses a single activation function, **GELU** (Gaussian Error Linear Unit), instead of ReLU. GELU is a smoother alternative to ReLU. Its formula is $GELU(x) = x \cdot \Phi(x)$, where $\Phi(x)$ is the cumulative distribution function (CDF) of the standard normal distribution.

These changes collectively show that architectural details matter significantly, and many "standard" practices can be revised to achieve better performance.

11.4 Upsampling and Transpose Convolutions

Many vision tasks, such as semantic segmentation, require the output to be an image of the same (or higher) resolution as the input. This requires **upsampling** operations to reverse the effect of downsampling (pooling, strided convolutions).

11.4.1 Upsampling Methods

- **Max Unpooling:** The reverse of max pooling. Requires storing the indices of the maxima from the forward pass to place the values back in their original locations. Other locations are filled with zeros.
- **Duplication / Nearest Neighbor:** A simple method where each low-resolution pixel value is copied to fill a block of pixels in the high-resolution output.
- **Learnable Interpolation:** The most powerful approach is to learn the upsampling function. This is typically achieved by:
 1. Perform a simple upsampling by inserting zeros between the pixels of the low-resolution input (zero-fill).
 2. Apply a standard learnable convolution to this sparse input. The convolution learns to "fill in the blanks" by interpolating from the non-zero values.

11.4.2 Transpose Convolution

A **transpose convolution** is a computational optimization for the "zero-fill + convolution" upsampling method. It is *not* a conceptually new operation.

- **Analogy:**
 - A **strided convolution** fuses a standard convolution and a subsequent subsampling step into one efficient operation.
 - A **transpose convolution** fuses a zero-fill upsampling step and a subsequent standard convolution into one efficient operation.
- **Mechanism:** It avoids the explicit creation of a large, zero-filled tensor and skips the wasteful multiplications by zero.
- **Nomenclature:** It is called "transpose" because if the forward pass of a standard convolution is represented as a matrix multiplication $y = Cx$, the corresponding transpose convolution's forward pass can be represented as $z = C^T y$. It is sometimes misleadingly called a "deconvolution," but it does not truly invert the convolution.

Conceptually, there are only standard convolutions. Strided and transpose convolutions are just efficient implementations for downsampling and upsampling, respectively.

11.5 Architectures for Image-Level Tasks: U-Nets

For tasks like semantic segmentation (classifying every pixel), a common architecture is an **encoder-decoder** network.

- **Encoder:** A contracting path, typically a standard ConvNet (like a ResNet), that progressively downsamples the input image to extract high-level semantic features (the "what").
- **Decoder:** An expansive path that progressively upsamples the feature maps using transpose convolutions to reconstruct a full-resolution output map (the "where").

11.5.1 The Bottleneck Problem

A simple encoder-decoder architecture forces all information to pass through a low-resolution bottleneck between the two paths. This makes it very difficult for the decoder to reconstruct fine-grained spatial details that were lost during downsampling.

11.5.2 The U-Net Solution

The **U-Net** architecture solves this problem by introducing long-range skip connections between the encoder and decoder.

- **Mechanism:** The feature maps from an encoder stage are concatenated channel-wise with the feature maps at the corresponding stage in the decoder (after upsampling).

- **Benefit:** This provides the decoder with direct access to high-resolution, low-level feature information from the encoder. The decoder can then combine the high-level semantic information (from the deeper layers) with this fine-grained spatial information to produce a much more precise output.
- **Structure:** The architecture diagram resembles the letter 'U', giving it its name. These long-range skip connections are used *in addition to* the short-range residual connections within each block of the encoder and decoder.

U-Net-like structures are fundamental to almost any task that requires generating high-resolution image outputs, including generative models.

Lecture 12: Graph Neural Nets

12.1 Introduction and Resources

This lecture transitions from Convolutional Neural Networks (CNNs) to Graph Neural Networks (GNNs). GNNs generalize the concept of local structure, moving from the rigid grid of an image to the more flexible structure of a graph.

Key Resources

- **Textbook:** Chapter 13 in "Computer Vision: Models, Learning, and Inference" by Simon Prince.
- **Blog Post:** "A Gentle Introduction to Graph Neural Networks" (Distill, 2021).
- **Software Packages:**
 - PyTorch Geometric (PyG)
 - Deep Graph Library (DGL)
 - Spektral (for TensorFlow)
- **Further Study:** Stanford's CS224W course on Machine Learning with Graphs.

12.2 Recap and A Loose End from CNNs

From a pedagogical perspective, understanding GNNs helps clarify which concepts from CNNs are specific to grid-like data and which are more general principles of network architecture.

Stochastic Depth Regularization

Before diving into GNNs, we address one final concept from the CNN module: stochastic depth regularization. This is a form of regularization analogous to Dropout, but applied at the block level in residual networks (ResNets).

- **Idea:** During training, randomly drop entire residual blocks with some probability p . The block is replaced by its identity skip connection.
- **Purpose:**
 - **Regularization:** Helps prevent overfitting.
 - **Functional Redundancy:** Similar to Dropout, it prevents any single block from becoming overly specialized. It encourages information to be distributed across multiple paths in the network.
- **Computational Advantage:** Unlike Dropout in MLPs, dropping an entire block yields a direct computational saving during training, as the forward and backward passes for that block are skipped.

- **Usage:** Typical dropout probabilities are around 10%. Schedules can vary, sometimes dropping earlier layers more frequently than later ones. It is a technique to consider when dealing with overfitting in deep ResNet-style architectures.

12.3 Generalizing CNNs to Graphs

The progression of architectures in this course has been from MLPs (fully connected, no assumed input structure) to CNNs (exploiting local grid structure). GNNs are the next step, designed for data where relationships are best represented by a graph.

Why Graphs?

Graphs are a general way to represent local structure and relationships. They appear in numerous applications:

- **Social Networks:** Users as nodes, connections (friendship, follows) as edges.
- **Citation Networks:** Papers as nodes, citations as directed edges.
- **Chemistry and Materials Science:** Atoms as nodes, chemical bonds as edges. Molecules are naturally graphs.
- **Physics and Engineering:** Meshes for 3D modeling, circuit diagrams, and physical systems.
- **Computer Vision:** Scene graphs representing objects and their spatial or semantic relationships.

Analogies Between CNN and GNN Tasks

We can draw direct parallels between common tasks for images and graphs.

- **Image Classification → Graph Classification:** The entire input (image or graph) is processed to produce a single label. For example, classifying a molecule graph based on a chemical property (e.g., "is it smelly?").
- **Semantic Segmentation → Node Classification:** Each component (pixel or node) of the input is assigned a label.
- **Other Tasks:** GNNs can also perform edge-level tasks, such as edge classification or link prediction (predicting if an edge should exist). This lecture will focus on node-level processing.

12.4 A Conceptual Bridge: Viewing an Image as a Graph

To understand how to build a GNN, we first reinterpret a CNN's operation through the lens of a graph. Let's consider an image as a specific type of graph.

- **Nodes:** Each pixel in the image is a node in the graph. The value(s) of the pixel (e.g., RGB vector) are the initial features of that node.
- **Edges:** Edges represent the local neighborhood structure. For a pixel, we can draw edges to its 8 neighbors.

- **Graph Properties:**

- **Directed Graph:** The relationship is directional. The pixel to the north of me has a "North" relationship to me, and I have a "South" relationship to it. These are distinct.
- **Labeled Edges:** Each edge has a label from a finite set, corresponding to its spatial direction: $\{N, S, E, W, NE, NW, SE, SW\}$.
- **Self-Loops:** Each node has a special relationship to itself.

A 3x3 Convolution as Message Passing

With this graph perspective, a 3x3 convolution can be seen as a message passing and aggregation step. A central node (pixel) updates its feature vector by receiving "messages" from its neighbors.

The computation for the output feature vector $\vec{h}_{out}$ at a single node is:

$$\vec{h}_{out} = \sigma \left(\vec{b} + W_{\text{self}} \vec{h}_{\text{in, self}} + \sum_{j \in \text{neighbors}} W_{\text{label}(j)} \vec{h}_{\text{in}, j} \right) \quad (1)$$

where:

- $\vec{h}_{\text{in}, j}$ is the input feature vector of neighbor j .
- $W_{\text{label}(j)}$ is a learnable weight matrix associated with the label of the edge from neighbor j (e.g., $W_N, W_S, \dots$).
- W_{self} is a weight matrix for the node's own information.
- $\vec{b}$ is a learnable bias vector.
- σ is a non-linear activation function.

This is mathematically identical to a 3x3 convolution. This re-framing gives us an immediate recipe to generalize CNNs to any directed graph with a finite set of edge labels.

What Generalizes Immediately?

Many core concepts from CNNs carry over directly to this GNN formulation.

- **Weight Sharing:** The same set of weight matrices ($W_N, W_S, \dots$) is used for every node in the graph, just as a single convolutional filter is applied across all spatial locations in an image.
- **Multiple Channels:** Node features can be vectors, corresponding to channels. The weight matrices W project from input channels to output channels.
- **1x1 Convolutions:** These correspond to applying an MLP-like transformation to each node's feature vector independently.
- **Residual Connections:** The output of a GNN layer can be added to its input.
- **Dropout for MLPs / Stochastic Depth:** These regularization techniques can be applied just as in CNNs.

What Does Not Generalize?

- **Pooling / Downsampling / Upsampling:** These operations are problematic. While down-sampling an image to a lower resolution is a natural concept, there is no universal, application-independent way to define a "lower-resolution" version of an arbitrary graph. This means most GNN architectures maintain the same graph structure throughout all layers.

What Needs Discussion: Normalization

Normalization layers need to be adapted to the graph structure.

- **Batch Normalization:** Difficult to apply directly. In a typical batch, different graphs may have different numbers of nodes, so the data tensor is not a simple, uniform block.
- **Layer Normalization:** A feasible option. Normalization statistics (mean, variance) can be computed over all channels and all nodes within a single graph. This assumes the entire graph fits into memory.
- **Per-Embedding (Instance) Normalization:** The simplest and most common choice. Each node's feature vector (embedding) is normalized independently over its channel dimension. The same normalization function (with shared learnable parameters, if any) is applied at each node.

12.5 Handling General Graph Structures

The "image-as-graph" model covers directed graphs with finite edge labels. However, many real-world graphs are different, most notably **undirected graphs with no edge labels**.

A Failed First Attempt

A naive approach is to reduce this case to the one we know.

1. Treat each undirected edge as two directed edges.
2. Assign a single, dummy label ("edge") to all edges.

Following our convolution-as-message-passing formula, the update rule becomes:

$$\vec{h}_{out} = \sigma \left(\vec{b} + W \sum_{j \in \text{neighbors}} \vec{h}_{in,j} \right) \quad (2)$$

The single weight matrix W can be pulled out of the sum due to linearity. This means the network can only sum (or average) the feature vectors of its neighbors and then apply a single linear transformation. After a few layers, this repeated averaging causes the features of all connected nodes to converge to a single value. This phenomenon is known as **over-smoothing**, and it severely limits the expressive power of the GNN.

A Principled Approach: Permutation Invariance

To overcome over-smoothing, we must design an architecture that respects the fundamental property of an unlabeled graph: a node's neighborhood is a **set**, not an ordered list. The aggregation function must be **permutation-invariant**.

A general form for a GNN layer respecting this constraint is:

$$\vec{h}_{out} = f_{W_1} \left(\vec{h}_{in, self}, \bigoplus_{j \in \text{neighbors}} g_{W_2}(\vec{h}_{in, j}) \right) \quad (3)$$

where $\bigoplus$ is a permutation-invariant aggregation operator.

Choices for Permutation-Invariant Aggregation There is a rich vocabulary of operations that do not depend on the order of their inputs:

- **Simple aggregators:** Sum, Mean, Max, Min, Product.
- **Statistical measures:** Median, Variance, general order statistics.
- **Machine Learning models:** Any ML model that operates on a set of data points is permutation-invariant. This includes powerful options like Least Squares or even kernel methods.
- **Attention-based mechanisms:** Using Softmax to create weighted combinations.

The need to handle this constraint in GNNs spurred creativity and led to the development of more powerful and general aggregation mechanisms.

12.6 The Attention Mechanism in GNNs

A more expressive and general architecture allows the self-node's features to influence how neighbor information is aggregated. A highly effective way to structure this is through an **attention mechanism**. This is motivated by the idea of using a learned similarity to weight the importance of different neighbors.

A general formulation inspired by kernel methods is:

$$\vec{h}_{out} = f \left(\vec{h}_{in, self}, \sum_{j \in \text{neighbors}} S(\vec{h}_{in, self}, \vec{h}_{in, j}) \cdot g(\vec{h}_{in, j}) \right) \quad (4)$$

Here, S is a learned similarity function. To control the dynamic range of the sum, we can frame it as an expectation. This is achieved by normalizing the similarity scores into probabilities using a **softmax** function.

An Example GNN Block with Attention

Let's assemble a complete GNN block using these design principles, wrapped in a residual connection for stable training.

For each node with input embedding $\vec{h}_0$ and neighbor embeddings $\{\vec{h}_i\}$:

1. **Projections (Query, Key, Value):** Project the embeddings using learnable linear layers (matrices P_q, P_k, P_v).

$$\text{query: } \vec{q} = P_q \vec{h}_0$$

$$\text{keys: } \vec{k}_i = P_k \vec{h}_i$$

$$\text{values: } \vec{v}_i = P_v \vec{h}_i$$

2. **Compute Attention Scores:** Calculate a similarity score between the self-node's query and each neighbor's key, often using a simple dot product.

$$s_i = \vec{q}^T \vec{k}_i \quad (5)$$

3. **Normalize Scores to Weights:** Apply softmax over the scores of all neighbors to get attention weights α_i that sum to 1.

$$\vec{\alpha} = \text{softmax}(\vec{s}) \quad (6)$$

4. **Aggregate Values:** Compute a weighted sum of the neighbor values using the attention weights. This is the aggregated neighborhood information.

$$\vec{h}_{\text{agg}} = \sum_{i \in \text{neighbors}} \alpha_i \vec{v}_i \quad (7)$$

5. **Update Node Representation:** The aggregated information is passed through a final transformation (e.g., an MLP) and added to the original input via a residual connection.

$$\vec{h}_{\text{out}} = \vec{h}_0 + \text{MLP}(\vec{h}_{\text{agg}}) \quad (8)$$

This entire process of computing query-key scores, applying softmax, and taking a weighted sum of values is the core of the **attention mechanism**. The necessary non-linearity to avoid over-smoothing comes from the softmax function and the multiplications within the attention computation and the final MLP.

Connection to Transformers

This attention mechanism, developed here from the constraints of operating on general graphs, is the fundamental building block of the **Transformer** architecture. A Transformer can be viewed as a GNN operating on a **complete graph**, where every node (e.g., a word in a sentence) is connected to every other node and attends to them to build a context-aware representation.

Lecture 13: DiffPool and finish GNNs, Start RNNs

13.1 Administrative Notes

13.1.1 Reading Material

- The material on Graph Neural Networks (GNNs) corresponds to Chapter 13 in the Prince textbook (*Understanding Deep Learning*).
- The content in this lecture, particularly on GNN pooling and Recurrent Neural Networks (RNNs), is not covered in the Prince textbook.
- For classic RNN material, a good (though older) reference is the online textbook at deeplearningbook.org. The perspectives in this course will be more modern.

13.1.2 Project Information

Key project deadlines and milestones will be announced, but here are the highlights:

- **Team Formation:** Groups of four.
- **Proposal Due:** November 3rd. Details on the proposal format will be provided.
- **Required Meetings:** Every project group must sign up for a meeting slot with the teaching staff in the week leading up to the proposal deadline to discuss their ideas.
- **Feedback Meetings:** Another meeting will occur the week after the proposal submission to get feedback from the TAs.
- **Draft Report:** A draft report will be due the week before Thanksgiving.
- **Peer Review:** You will be expected to conduct peer reviews of other teams' draft reports.
- **Final Report:** The final project report is due on the last day of class.
- **Poster Presentation:** A poster presentation will take place during RRR week.

13.2 Finishing Graph Neural Networks (GNNs)

We previously generalized Convolutional Neural Networks (CNNs) to operate on graphs. We established a framework for directed graphs with a finite set of edge labels, where CNN ideas translate directly. The key remaining challenge is handling pooling and downsampling, which does not have a straightforward generalization, especially for undirected graphs with no edge labels.

13.2.1 The Challenge of Pooling and Downsampling in GNNs

Why Do We Pool Anyway? The primary motivation for pooling and downsampling in CNNs is to **grow the receptive fields faster with each layer**. This allows the network to capture attributes of the input (e.g., an image) that exist at a larger spatial scale. Without pooling, a deep stack of small convolutions (e.g., 3x3) would be needed to achieve a large receptive field, which is computationally inefficient and may not capture the desired inductive bias.

This motivation is still highly relevant for GNNs. In a graph, information from distant nodes propagates slowly through message passing, one hop per layer. To make decisions based on global graph structure, we need an effective way to aggregate information from distant parts of the graph quickly.

A Simple Hack: The Global Node A straightforward, albeit blunt, solution to accelerate information flow is to introduce a **fake global node**.

- This node is added to the graph and connected to every other node.
- **Step 1:** In one layer of message passing, all nodes send their information to this global node.
- **Step 2:** The global node aggregates this information. In the next layer, it broadcasts the aggregated information back to all other nodes.

After two layers, every node has access to information from the entire graph, effectively making its receptive field global.

Drawbacks:

- **Coarseness:** This method is an all-or-nothing approach. It provides access to local structure (immediate neighbors) and global structure, but not intermediate, multi-hop structures that might be important.
- **Oversquashing:** This introduces a significant information bottleneck. If the graph has thousands of nodes, all of their information must be compressed (“squashed”) into the single, finite-dimensional embedding of the global node. This can lead to a substantial loss of detail. Oversquashing is a general challenge in GNNs where information from many nodes must be funneled through a smaller-dimensional representation.

Note: Oversquashing is distinct from *oversmoothing*, where repeated averaging operations cause node representations to become indistinguishable.

13.2.2 A Principled Approach: Differentiable Pooling (DiffPool)

To create a more nuanced pooling mechanism, we need to go from many nodes to fewer nodes based on the graph’s structure and the nodes’ vector contents. This problem is analogous to **clustering**.

The Clustering Analogy Classic clustering algorithms like k-means (Lloyd’s algorithm) perform a similar task. However, k-means involves a step that is not differentiable: the hard assignment of each point to its nearest cluster center. This assignment is like a step function; a small change in a center’s position can cause a point to abruptly switch clusters, which means the derivative is not well-defined.

Making Clustering Differentiable with Soft Assignments To overcome the non-differentiability of hard assignments, we can use an idea similar to softmax: **soft assignments**.

- Instead of assigning each node to a single cluster, we assign it to *every* cluster with a certain probability.
- Each node will have a probability distribution over the new, coarsened "cluster-nodes".
- A consequence of this is that the resulting downsampled graph will be fully connected, with edges weighted by the probabilistic assignments.

This approach is detailed in the paper "Hierarchical Graph Representation Learning with Differentiable Pooling" by Ying et al., 2018.

The DiffPool Module: A GNN for Pooling Since we don't know the optimal way to cluster nodes for a given task, we follow the deep learning principle: **make it learnable**. We design a learnable module (a "grey-box") that performs this soft clustering.

Module Specification

- **Input:** The current graph with node embeddings. Let's say we have N nodes and are at layer l , with embeddings $H^{(l)} \in \mathbb{R}^{N \times d}$.
- **Output:** A probability vector for each node, indicating its membership in k new clusters. This can be represented by an assignment matrix $S \in \mathbb{R}^{N \times k}$, where S_{ij} is the probability that node i belongs to cluster j . Each row of S sums to 1.

Implementation The task of generating these assignment vectors is essentially a **node-level classification problem** on the graph. The most powerful tool we have for learnable tasks on graphs is a GNN itself. Therefore, the DiffPool module is implemented as a **separate, smaller GNN**.

1. This "pooling GNN" takes the current node embeddings $H^{(l)}$ as input.
2. It performs several layers of message passing to produce new embeddings that are suitable for the clustering task.
3. A final linear layer followed by a 'softmax' operation on each node's output generates the desired assignment matrix S .

The parameters of this pooling GNN are trained jointly with the main GNN.

Constructing the Coarsened Graph Once the assignment matrix S is computed, we can construct the new, smaller graph with k nodes.

1. New Node Embeddings The embedding for each of the k new cluster-nodes is a weighted average of the original node embeddings, where the weights are the assignment probabilities. The content of the original nodes is "poured" into the new clusters.

$$H^{(l+1)} = S^T H^{(l)}$$

Here, $H^{(l+1)} \in \mathbb{R}^{k \times d}$ are the embeddings for the k new nodes.

2. New (Soft) Adjacency Matrix We also need a new adjacency matrix $A^{(l+1)} \in \mathbb{R}^{k \times k}$ for the coarsened graph. This matrix should represent the connection strength between the new cluster-nodes. It can be computed as:

$$A^{(l+1)} = S^T A^{(l)} S$$

The entry $A_{ij}^{(l+1)}$ represents the total strength of connections between the nodes that constitute cluster i and the nodes that constitute cluster j . This results in a dense, weighted adjacency matrix. Our GNN layers must be able to handle these scalar edge weights (e.g., by multiplying messages by the edge weight).

Training with Auxiliary Losses While DiffPool can be trained end-to-end via the main task loss, this can be difficult. The pooling GNN starts with random weights and may produce poor clusterings, leading to noisy gradients. To guide the training, we introduce **auxiliary losses** that encourage desirable properties in the clustering. These losses are added to the main loss during training.

Property 1: Clusters should respect the graph topology. Nodes that are connected in the original graph should be likely to be assigned to the same cluster. This can be encouraged with a **link prediction loss**.

$$\mathcal{L}_{\text{link}} = \|A^{(l)} - SS^T\|_F^2$$

Intuition: The matrix $A^{(l)}$ represents the graph's connectivity. The matrix $SS^T \in \mathbb{R}^{N \times N}$ is a low-rank approximation of connectivity based on cluster assignments. The (i, j) -th entry of SS^T is the inner product of the assignment vectors for nodes i and j . This loss encourages connected nodes (where $A_{ij} = 1$) to have similar assignment vectors (large inner product), thus placing them in the same cluster. This helps the GNN find community structures.

Property 2: Cluster assignments should be close to hard assignments. We introduced soft assignments for differentiability, but we would prefer if each node primarily belongs to one cluster. This can be encouraged by penalizing high entropy in the assignment vectors.

$$\mathcal{L}_{\text{entropy}} = \frac{1}{N} \sum_{i=1}^N H(\vec{s}_i)$$

where $H(\cdot)$ is the entropy function and $\vec{s}_i$ is the i -th row of S . Entropy is minimized when a probability distribution is a one-hot vector (i.e., a hard assignment). This loss encourages the softmax outputs to be sharp and decisive.

These auxiliary losses are combined with the main task loss, usually with hyperparameters to weight their contribution. The gradients from all losses flow back to update both the main GNN and the pooling GNN.

13.3 Final GNN Challenge: Handling Held-Out Data

In standard machine learning, it is crucial to have held-out data for validation and testing. For node-level tasks on a single, large graph (e.g., a social network), we cannot simply hold out an entirely separate graph. We must hold out parts of the single graph we possess.

13.3.1 Approach 1: Excise Entire Nodes

One method is to completely remove a set of nodes and their corresponding edges from the graph during training. These nodes are then used for testing.

- **Challenge:** This fundamentally **alters the graph's connectivity**. Removing nodes can break important information pathways for the remaining nodes. For structured graphs like molecules, removing an atom changes the identity of the molecule itself.

13.3.2 Approach 2: Mask Node Content

A better approach is to remove the *content* (feature vector) from a node but leave the node's structure (its connections) intact within the graph.

- **Problem:** What do we replace the feature vector with? We cannot use a null value, as the network's operations require a vector.
- **Solution:** Make it learnable. We replace the features of any masked node with a single, shared, learnable vector, often called a **mask token**.

Training Process:

1. A learnable vector, $\vec{v}_{\text{mask}}$, is initialized.
2. During training, we perform data augmentation: we randomly select some nodes from the training set, replace their feature vectors with $\vec{v}_{\text{mask}}$, but *keep* the loss calculation on their original labels.
3. The network learns to use the information from a masked node's neighbors to make a correct prediction for that node. In doing so, it learns both a meaningful representation for $\vec{v}_{\text{mask}}$ and how to process it.
4. For the actual held-out (test) nodes, we replace their features with this learned $\vec{v}_{\text{mask}}$ and evaluate the model's predictions.

13.4 Introduction to Recurrent Neural Networks (RNNs)

13.4.1 Historical Context

Before 2018, the deep learning landscape was largely dominated by two architectures:

- **CNNs** for vision and other spatial data.

- **RNNs** for sequential data like language and speech.

In 2017, the paper "Attention Is All You Need" introduced the Transformer architecture, which quickly began to outperform RNNs on their home turf. For a time, RNNs were considered by many to be obsolete. However, recent state-of-the-art models have begun to re-incorporate ideas from the RNN lineage, showing their continued relevance.

13.4.2 The Core Idea: Sequential and Causal Data

The key characteristic of data like language, speech, and time-series is that it is **sequential**. It has a one-dimensional topology with a strict ordering and a natural **causal** flow: the present depends on the past, but not the future. RNNs are architectures designed specifically to model this structure.

13.4.3 A Signal-Processing Perspective

We can draw an analogy to classical signal processing filters:

- **CNNs** are analogous to **Finite Impulse Response (FIR) filters**. They perform weight sharing across **space**.
- **RNNs** are analogous to **Infinite Impulse Response (IIR) filters**. They maintain a finite-dimensional hidden state and perform weight sharing across **time**.

Our approach to understanding RNNs will be to adapt the design principles we learned from CNNs, but with two key changes:

1. Replace the notion of "space" with "time".
2. Enforce causality, meaning the computation at a given timestep can only depend on current and past inputs.

Lecture 14: RNNs and Self-Supervision

14.1 Introduction to Recurrent Neural Networks (RNNs)

14.1.1 Context and History

Recurrent Neural Networks (RNNs) represent a significant class of neural network architectures, historically parallel to Convolutional Neural Networks (CNNs) in their success. Before 2018, the consensus view held two major successes in deep learning:

- **CNNs** for images and computer vision.
- **RNNs** for language, speech, and other Natural Language Processing (NLP) tasks.

The key challenge that RNNs address is the processing of **sequential data**. This includes domains like language, speech, control systems, and time-series analysis, where data points arrive in a specific order and often exhibit causal relationships (i.e., later events depend on earlier ones).

14.1.2 A Signal-Processing Perspective

We can draw a useful analogy between neural network components and classical signal processing filters.

- **Finite Impulse Response (FIR) Filters:** These filters have a finite memory of past inputs. This concept maps directly to **CNNs**, where weight sharing occurs **across space**. The receptive field is finite and grows with network depth.
- **Infinite Impulse Response (IIR) Filters:** These filters, in principle, have an infinite memory of the past. For practical, tractable implementations, they are designed with a **finite-dimensional hidden state**. This concept of a filter with memory maps to **RNNs**. The crucial difference is that weight sharing occurs **across time**. These systems are fundamentally sequential and causal.

Inside a Linear IIR Filter A standard linear IIR filter operates on an input sequence $\vec{u}_t$ to produce an output sequence $\vec{y}_t$. It maintains a hidden state $\vec{h}_t$, which acts as a running summary of the relevant parts of the input history. The update process is defined by two linear equations:

1. **State Update (Causal):** The new hidden state $\vec{h}_{t+1}$ is a linear combination of the previous state $\vec{h}_t$ and the current input $\vec{u}_{t+1}$.

$$\vec{h}_{t+1} = W_h \vec{h}_t + B \vec{u}_{t+1} \quad (9)$$

Here, W_h and B are weight matrices. The hidden state is typically initialized to zero.

2. **Output Generation:** The output $\vec{y}_{t+1}$ is a linear combination of the current hidden state $\vec{h}_{t+1}$ and, optionally, a direct feed-through of the current input $\vec{u}_{t+1}$.

$$\vec{y}_{t+1} = C \vec{h}_{t+1} + D \vec{u}_{t+1} \quad (10)$$

Here, C and D are weight matrices. The dimensions of $\vec{u}$, $\vec{h}$, and $\vec{y}$ can all be different, with the matrices sized accordingly.

The core idea is to treat this IIR filter block as the sequential counterpart to a convolution layer in a CNN.

14.2 Designing Modern RNNs

14.2.1 Generalizing from CNN Principles

We adopt the design principles from CNNs and adapt them for sequential data. We replace space with time and enforce causality. The conventional visualization for an RNN architecture is as follows:

- **Horizontal Axis:** Represents time or the sequence order.
- **Vertical Axis:** Represents the depth of the network (layers).

Information flows from left to right (through time) and from bottom to top (through layers).

Many design ideas from CNNs carry over directly in the vertical (depth) dimension:

- **Vertical Residual Connections:** Skip connections can be added between layers at the same time step, just as in ResNets.
- **Vertical "1x1" MLPs:** Multi-Layer Perceptrons can be applied to the feature vector at each time step independently, analogous to 1x1 convolutions.
- **Vertical Normalizations:** Normalization schemes like LayerNorm or RMSNorm can be applied to the feature vector at a specific time step. It's crucial that these normalizations do not "look into the future"; they operate only on the local channels at the current time step.

14.2.2 Traditional "Vanilla" RNNs

A key design question is: **Where do the non-linearities go?** To achieve expressive power, non-linearities are essential.

The traditional RNN perspective places a non-linearity in the state update mechanism. We start with the linear dynamics and modify them:

1. We can drop the feed-through term $D\vec{u}_{t+1}$ from the core RNN block, as this can be implemented as a parallel skip connection if needed.
2. We add learnable biases, a standard practice in neural networks.
3. We wrap the linear state update inside a non-linear activation function σ .

This yields the state update equation for a traditional RNN cell:

$$\vec{h}_{t+1} = \sigma(W_h \vec{h}_t + B\vec{u}_{t+1} + \vec{b}_h) \quad (11)$$

The output of an RNN layer is typically just the hidden state itself, $\vec{y}_{t+1} = \vec{h}_{t+1}$. If further transformation is needed, an explicit linear layer can be added on top.

Choice of Non-Linearity Unlike in CNNs where ReLU and its variants dominate, classical RNNs typically use saturating non-linearities like **sigmoid** or **tanh**. The reason for this choice relates to the problem of exploding activations. Since the recurrent update is applied repeatedly over many time steps, an expansive mapping (like ReLU) can cause the state vectors to grow without bound. Saturating functions constrain the activations, helping to maintain stability.

14.3 Training RNNs: Backpropagation Through Time (BPTT)

14.3.1 Gradient Flow

Training requires a loss function. Depending on the task, a loss can be computed at every time step (e.g., sequence tagging) or only at the final time step (e.g., sequence classification).

During backpropagation, gradients flow backward through the computation graph:

- **Vertically:** Gradients flow down through the layers at each time step, similar to a standard feed-forward network.
- **Horizontally:** Gradients also flow backward in time, from $\vec{h}_{t+1}$ to $\vec{h}_t$, through the recurrent connection. This unrolling of the network through time is why the process is called Backpropagation Through Time.

This horizontal gradient path can become extremely long for sequences with many time steps.

14.3.2 The Problem of Long-Range Dependencies

The long path for gradient propagation in time leads to a significant challenge:

- **Vanishing Gradients:** For the network to be stable, the recurrent dynamics must generally be contractive. This means that during backpropagation, gradients are repeatedly multiplied by the recurrent weight matrix. For a stable system, this causes gradients to shrink exponentially as they flow back in time. As a result, the influence of early time steps on the loss at a much later time step becomes negligible, making it difficult for the network to learn long-range dependencies.
- **Exploding Gradients:** Conversely, if the dynamics are expansive, gradients can grow exponentially, leading to unstable training. This is mitigated by saturating non-linearities and techniques like gradient clipping.

Historically, architectures like **LSTMs (Long Short-Term Memory)** and **GRUs (Gated Recurrent Units)** were developed to address the vanishing gradient problem. They introduced complex gating mechanisms to better control the flow of information and gradients over long time scales. These were highly effective but also very delicate and complex.

14.3.3 A Modern View: Parallelism during Training

A critical issue with the traditional RNN structure is its inherent sequentiality, which makes it inefficient to train on modern parallel hardware (GPUs/TPUs).

- **Dependency Graph:** To compute the state at layer l and time t , one must have already computed the state at layer $l - 1$ at time t AND the state at layer l at time $t - 1$.
- **Limited Parallelism:** This creates a diagonal "wavefront" of computation. The maximum number of operations that can be performed in parallel at any given moment is limited by the number of layers in the network, not the length of the sequence. This is a form of "pipeline parallelism" and is often a poor utilization of hardware resources for long sequences.

The fundamental bottleneck to parallelization is the **non-linearity in the recurrent (horizontal) path**. If the recurrence were purely linear, various algebraic techniques could be used to parallelize the computation.

14.4 Example: Learning a Kalman Filter

To build intuition, we consider a purely linear system. The **Kalman Filter (KF)** is the optimal estimator for the state of a linear dynamical system with Gaussian noise.

14.4.1 Problem Setup

Consider an underlying true linear process with state $\vec{x}_t$, input $\vec{u}_t$, and output $\vec{y}_t$.

$$\vec{x}_{t+1} = A\vec{x}_t + B\vec{u}_t + \vec{w}_t \quad (\text{Process dynamics}) \quad (12)$$

$$\vec{y}_{t+1} = C\vec{x}_{t+1} + D\vec{u}_{t+1} + \vec{v}_{t+1} \quad (\text{Observation model}) \quad (13)$$

where $\vec{w}_t$ and $\vec{v}_t$ are i.i.d. zero-mean Gaussian noise. If all system matrices (A, B, C, D) and noise covariances are known, the steady-state Kalman Filter provides the optimal estimate $\hat{\vec{x}}_t$ of the true state $\vec{x}_t$. The KF itself is a linear recurrent system:

$$\hat{\vec{x}}_{t+1} = \tilde{A}\hat{\vec{x}}_t + \tilde{B}\vec{u}_{t+1} + F\vec{y}_{t+1} \quad (14)$$

This is a linear RNN.

14.4.2 Case 1: Supervised Learning

Suppose we do not know the system matrices, but we have a training dataset of input/output traces $\{(\vec{u}^{(j)}, \vec{y}^{(j)})_t\}$ along with the corresponding **ground truth hidden states** $\{\vec{x}_t^{(j)}\}$.

1. We can define a linear RNN with learnable parameter matrices $\tilde{A}, \tilde{B}, F$.
2. We can define a supervised loss function, such as the mean squared error between our estimate and the true state: $\mathcal{L} = \sum_{j,t} \|\hat{\vec{x}}_t^{(j)} - \vec{x}_t^{(j)}\|^2$.
3. We can train this RNN using gradient descent and BPTT to learn the optimal filtering matrices.

14.4.3 Case 2: The Need for Self-Supervision

Now, consider a more realistic scenario: what if we only have the input/output traces $\{(\vec{u}^{(j)}, \vec{y}^{(j)})_t\}$ and **no access to the true hidden states** $\vec{x}_t$?

Question: Can we learn the true Kalman Filter?

Answer: No. The problem is fundamentally underdetermined. The internal state representation $\vec{x}$ is not uniquely identifiable from input-output data alone.

- **Unobservability through Change of Coordinates:** Any invertible linear transformation (change of basis) applied to the state space, $\vec{z} = T\vec{x}$, would result in a different set of system matrices (A', B', C') that produce the *exact same* input-output behavior.
- Since we cannot distinguish between these different coordinate systems from the data, we cannot learn the "true" state $\vec{x}$.
- However, some properties of the system, like the eigenvalues of the dynamics matrix A , are invariant to a change of basis. This suggests we can learn the *essence* or *structure* of the dynamics, but not the specific coordinate system.

This motivates the need for a different learning paradigm where we do not rely on external labels.

14.5 Self-Supervision

14.5.1 General Principle

Self-supervision is a learning paradigm that bridges supervised and unsupervised learning. The core idea is:

I need labels to train, but I don't have them. So, I will **make my own labels from the data itself**.

We design a "pretext task" where one part of the data is used to predict another part of the same data. This allows us to use the machinery of supervised learning (loss functions, backpropagation) without requiring external annotations.

14.5.2 Self-Supervised Learning for the Filter Problem

Since we cannot predict the unobserved state $\vec{x}_t$, we must choose a target that is available in our data. For a causal system, a natural choice is to predict the future.

- **Pretext Task:** Predict the next observation, $\vec{y}_{t+1}$, using the history of inputs and outputs up to time t .
- **Model:** We use our linear RNN to produce a hidden state $\hat{\vec{x}}_t$, which summarizes the past.
- **Scaffolding:** Since the state $\hat{\vec{x}}_t$ may have a different dimension than the observation $\vec{y}_{t+1}$, we add an extra component, a "prediction head" (e.g., a linear layer), to map the state to a prediction of the output. This is temporary "scaffolding" used only during training.

- **Loss:** The loss is the error between our prediction and the actual next observation from the data: $\mathcal{L} = \sum_{j,t} \|\text{predicted}(\hat{x}_t^{(j)}, \vec{u}_{t+1}^{(j)}) - \vec{y}_{t+1}^{(j)}\|^2$.

By training on this task, the network is forced to learn an internal representation ($\hat{x}_t$) that captures the essential dynamics of the system, sufficient for predicting future outputs. This learned representation captures the system's structure up to the unobservable invariances (like the change of coordinates).

14.5.3 Key Lessons from the Example

1. We can learn a useful **partial pattern** even if the full system is not identifiable.
2. We may need **scaffolding** (e.g., prediction heads) to formulate a trainable loss. These components can be discarded after the primary representation is learned.
3. For causal sequences, **next-step prediction** is a generic and powerful self-supervision strategy.

14.6 Connection to Unsupervised Learning

Self-supervision is the modern, deep-learning approach to the classical machine learning problem of **unsupervised learning**. The goal is to discover intrinsic structure in unlabeled data. Classical approaches include:

1. **Dimensionality Reduction** (e.g., PCA): Assumes data lies on a lower-dimensional manifold.
2. **Clustering** (e.g., K-Means): Assumes data is organized into distinct groups or blobs.

14.6.1 Thinking about PCA as Self-Supervision

Principal Component Analysis (PCA) is typically presented as an algorithm (compute covariance, find eigenvectors). Can we reframe it in the language of self-supervision?

- The **Eckart-Young-Mirsky theorem** states that the best rank- k approximation of a matrix X (in terms of Frobenius norm) is given by its truncated Singular Value Decomposition (SVD).

$$\hat{X} = \arg \min_{\text{rank}(Z) \leq k} \|X - Z\|_F^2 \quad (15)$$

- This provides a reconstruction loss! We can interpret PCA as trying to find a low-dimensional representation from which the original data can be best reconstructed.
- This framing naturally leads to the concept of an **autoencoder**, a neural network trained to reconstruct its own input after passing it through an information bottleneck. This provides a way to learn dimensionality reduction using gradient-based optimization.

Lecture 15: Finish Self-Supervision and Start State-Space Models

15.1 Self-Supervision

15.1.1 General Principles

The core idea of self-supervision addresses a fundamental problem in machine learning: training algorithms require labels, but labeled data is often scarce.

“I need labels to train, I don’t have labels, so I make my own labels from data.”

This approach allows us to learn meaningful patterns and representations directly from unlabeled data. From the previous lecture’s Kalman filtering example, where we tried to learn an underlying state without ground truth labels, we learned several lessons:

- **Learning Partial Patterns:** It is possible to learn useful, albeit partial, patterns. In the Kalman filter case, we could learn the system dynamics up to a change of coordinates.
- **Scaffolding:** We often need to create temporary or “scaffolding” parts of a neural network (like an artificial output layer) to define a loss function for training. These parts may be discarded after training.
- **Next-Thing Prediction:** In causal sequence modeling, a generic and powerful self-supervision technique is to predict the next element in a sequence.

15.1.2 Connecting to Classical Unsupervised Learning

To better understand self-supervision, we can connect it to classical unsupervised learning methods. The two main approaches in classical ML are:

1. **Dimensionality Reduction** (e.g., Principal Component Analysis - PCA)
2. **Clustering** (e.g., K-Means)

Recalling PCA Given a dataset of unlabeled data points $\{\vec{x}_i\}_{i=1}^N$ where $\vec{x}_i \in \mathbb{R}^d$, PCA aims to find a lower-dimensional representation that captures the most variance in the data.

Classic Recipe for PCA:

1. Construct a data matrix $X \in \mathbb{R}^{N \times d}$, where each row is a data point $\vec{x}_i^T$.
2. Compute the Singular Value Decomposition (SVD) of X : $X = U\Sigma V^T$.
3. Keep the top k right singular vectors, $\vec{v}_1, \dots, \vec{v}_k$, which correspond to the largest singular values. These vectors form an orthonormal basis for the k -dimensional subspace of greatest variance.
4. For a new data point $\vec{x}$, its k -dimensional feature representation is obtained by projecting it onto this basis.

The Deep Learning Perspective: From a modern deep learning viewpoint, this classical recipe feels very different from what we are used to. Specifically, it has:

- No loss function
- No labels
- No gradients
- No optimizer
- No mini-batches

This motivates us to reinterpret PCA in a way that aligns with the deep learning framework.

15.1.3 Reinterpreting PCA: The Autoencoder

A Loss-Based View via Eckart-Young-Mirsky Theorem The Eckart-Young-Mirsky theorem provides a link between the SVD and a loss minimization problem. The theorem states that the best rank- k approximation $\hat{X}$ to a matrix X (in the sense of minimizing the Frobenius norm of the difference) is obtained by truncating the SVD of X at the k -th term.

$$\min_{\text{rank}(\hat{X}) \leq k} \|X - \hat{X}\|_F^2$$

The solution is $\hat{X} = \sum_{i=1}^k \sigma_i \vec{u}_i \vec{v}_i^T$. This gives us something that looks like a loss function: the mean squared error between the original data matrix and its low-rank reconstruction.

Formulating a Neural Network We can now translate this loss minimization problem into a neural network training problem. The goal is to learn a mapping that reconstructs the input vector $\vec{x}$ after passing it through a low-dimensional bottleneck.

- **Input and Label:** The input is a data vector $\vec{x} \in \mathbb{R}^d$. For self-supervision, the label is the input itself, $\vec{x}$.
- **Loss Function:** The reconstruction loss is the mean squared error: $\|\vec{x} - \hat{\vec{x}}\|_2^2$, where $\hat{\vec{x}}$ is the reconstructed vector.
- **Architecture:** To enforce the rank- k constraint, we design an architecture with a bottleneck. We can use two linear layers:
 1. An **encoder**, $W_1 \in \mathbb{R}^{k \times d}$, maps the d -dimensional input to a k -dimensional hidden representation (where $k < d$).
 2. A **decoder**, $W_2 \in \mathbb{R}^{d \times k}$, maps the k -dimensional representation back to the original d -dimensional space.

The reconstructed vector is $\hat{\vec{x}} = W_2 W_1 \vec{x}$. The product $W_2 W_1$ is inherently at most rank k .

This network can be trained over the entire dataset using standard deep learning techniques like mini-batch gradient descent. The learned encoder W_1 then serves the same purpose as the projection in PCA.

The Autoencoder This neural network architecture is called an **autoencoder**. Its core ingredients are:

- **Labels:** The inputs themselves ($\vec{x}_i$).
- **Architecture:** An encoder, a bottleneck (hidden layer), and a decoder.
- **Goal:** Learn an encoding that captures the most important information needed for reconstruction.

From a traditional perspective, the primary goal is to obtain the encoder for dimensionality reduction on new data. The decoder is merely **scaffolding** needed to define the loss and enable training.

15.1.4 Variations of Autoencoders

The idea of forcing information through a bottleneck to learn essential features can be generalized. We can encode the desired regularities in our representation using architecture, auxiliary losses, or data augmentation.

Classic (Vanilla) Autoencoder This is the architecture described above. The regularity (low-dimensionality) is enforced by the **architectural bottleneck**, where the hidden layer has fewer neurons than the input/output layers. The encoder and decoder can be arbitrary neural networks (e.g., CNNs for images), not just single linear layers.

Sparse Autoencoder Instead of an architectural bottleneck, we can impose a bottleneck using an **auxiliary loss**.

- The hidden layer can be larger than the input layer.
- We add a sparsity-seeking penalty to the loss function, such as an L1-norm penalty on the hidden activations.
- The total loss becomes: $L = \|\vec{x} - \hat{\vec{x}}\|_2^2 + \lambda \|\vec{h}\|_1$, where $\vec{h}$ is the hidden representation.
- The L1 penalty encourages most hidden units to be zero, forcing the network to use only a small subset of features to represent any given input. The information is in both the values of the non-zero activations and *which* activations are non-zero.

This encourages a representation where data is described by a combination of a few "basis" features from a large dictionary.

Denoising Autoencoder Here, the regularity is enforced through **data augmentation**.

- The input to the encoder is a corrupted version of the original data, $\tilde{\vec{x}}$ (e.g., $\vec{x}$ with added Gaussian noise).
- The network is trained to reconstruct the *original, clean* data $\vec{x}$ from the corrupted input $\tilde{\vec{x}}$.
- To succeed, the encoder must learn to capture the robust, essential features of the data manifold and ignore the noise.

- This prevents the network from learning a trivial identity mapping, even without an architectural bottleneck.

Masked Autoencoder This is another variant based on data augmentation, particularly popular in language (e.g., BERT) and vision.

- A random portion of the input is masked or removed (e.g., setting patches of an image to a gray value).
- The autoencoder's task is to reconstruct the full, original input from the partially masked version.
- This forces the model to understand the context and underlying structure of the data to successfully "fill in the blanks."
- The loss can be computed over the entire reconstructed image or, in some variants, only on the masked regions.

15.1.5 Contrastive Self-Supervision

Just as autoencoders connect to dimensionality reduction, we can find a self-supervision paradigm that connects to the spirit of **clustering**. This is called **contrastive self-supervision**.

Core Idea:

- An example and its own data augmentations should belong to the same cluster. In an embedding space, this means they should be close to each other. These are called **positive pairs**.
- Fundamentally different examples from the dataset should belong to different clusters. In an embedding space, this means they should be far from each other. These are called **negative pairs**.

Basic Structure:

1. Take a data point $\vec{x}_i$ (the "anchor").
2. Create a positive pair by applying a data augmentation to it: $\text{aug}(\vec{x}_i)$.
3. Select a different data point $\vec{x}_j$ as a negative pair.
4. Pass all three through an encoder to get embeddings: e_i , e'_i , and e_j .
5. Define a loss function that rewards closeness for the positive pair (pulls e_i and e'_i together) and penalizes closeness for the negative pair (pushes e_i and e_j apart).

This method learns an embedding space where similar items are clustered together, without needing a decoder. The reliance on negative samples can sometimes be problematic (e.g., pushing apart two different images of the same class), leading to advanced methods that work without explicit negative examples.

15.2 State-Space Models (SSMs)

15.2.1 Motivation: Problems with Traditional RNNs

Recurrent Neural Networks (RNNs) are designed for sequential data, but they suffer from significant drawbacks:

1. **Difficult to Parallelize Training:** The recurrence relation $h_t = f(h_{t-1}, x_t)$ is inherently sequential. To compute the hidden state at time t , one must first compute it at $t - 1$. This prevents parallelization across the time dimension during training.
2. **Difficulty Learning Long-Range Dependencies:** The repeated application of non-linearities and matrix multiplications during backpropagation can lead to vanishing or exploding gradients, making it hard to capture dependencies over long sequences.

15.2.2 The Key Idea: Eliminate Horizontal Non-Linearity

The core innovation of State-Space Models (SSMs) is to address these issues by making the recurrence **linear**. We eliminate the "horizontal" non-linearity that operates along the time axis.

A traditional RNN update looks like: $\vec{h}_{t+1} = \sigma(A\vec{h}_t + B\vec{x}_t + \vec{b})$. The new SSM approach simplifies this to a linear system:

$$\begin{aligned}\vec{h}_{t+1} &= A\vec{h}_t + B\vec{x}_{t+1} \\ \vec{y}_t &= C\vec{h}_t + D\vec{x}_t\end{aligned}$$

Here, A, B, C, D are learned weight matrices that do not depend on the input. This is a Linear Time-Invariant (LTI) system, a well-understood concept from control theory and signal processing. The vector $\vec{h}_t$ is the **state**, which acts as the model's memory, summarizing the history of the sequence up to time t .

15.2.3 The Convolution View for Parallel Training

By removing the non-linearity, we can unroll the recurrence to express the output $\vec{y}_k$ purely in terms of the inputs, without intermediate hidden states.

$$\begin{aligned}\vec{h}_0 &= B\vec{x}_0 \\ \vec{h}_1 &= A\vec{h}_0 + B\vec{x}_1 = AB\vec{x}_0 + B\vec{x}_1 \\ \vec{h}_2 &= A\vec{h}_1 + B\vec{x}_2 = A^2B\vec{x}_0 + AB\vec{x}_1 + B\vec{x}_2 \\ &\vdots \\ \vec{h}_k &= \sum_{i=0}^k A^{k-i}B\vec{x}_i\end{aligned}$$

Substituting this into the output equation gives:

$$\vec{y}_k = C\vec{h}_k + D\vec{x}_k = C \left(\sum_{i=0}^k A^{k-i} B\vec{x}_i \right) + D\vec{x}_k$$

This expression is a **convolution**. The output at each time step is a linear combination of all past inputs. The convolution kernel (or filter) is defined by the matrices $\{CA^k B\}_{k=0,1,\dots}$.

Since this is a convolution, the entire output sequence can be computed in parallel during training. We no longer need to compute hidden states sequentially.

15.2.4 Improving Computational Efficiency

From $O(T^2)$ to $O(T \log T)$ with FFT A naive implementation of this convolution for a sequence of length T requires $O(T^2)$ total computation. We can significantly speed this up using the **Fast Fourier Transform (FFT)**.

- The convolution theorem states that convolution in the time domain is equivalent to element-wise multiplication in the frequency domain.
- The FFT algorithm can compute the Discrete Fourier Transform of a sequence of length T in $O(T \log T)$ time.
- **Algorithm:**
 1. Compute the FFT of the input sequence $\vec{x}$.
 2. Compute the FFT of the convolution filter (derived from A, B, C).
 3. Multiply the results element-wise in the frequency domain.
 4. Compute the Inverse FFT to get the final output sequence $\vec{y}$.
- This reduces the overall training complexity to $O(T \log T)$, making SSMs highly efficient for long sequences.
- A technical challenge is that the inputs $\vec{x}_t$ are vectors, not scalars. The FFT machinery must be carefully adapted to handle these vector-valued sequences, typically by breaking the computation down component-wise.

Speeding up Kernel Computation with a Diagonal State Matrix Computing the convolution kernel itself involves calculating matrix powers A^k , which can be computationally intensive and adds depth to the computation graph. To simplify this, we can impose structure on the state matrix A .

- We restrict A to be a **diagonal matrix**.
- If $A = \text{diag}(\lambda_1, \dots, \lambda_n)$, then $A^k = \text{diag}(\lambda_1^k, \dots, \lambda_n^k)$, which is trivial to compute.
- This might seem restrictive, but many matrices are diagonalizable. By learning the parameters, the model can effectively learn a basis in which the system dynamics are diagonal, capturing a wide range of behaviors.

15.2.5 Expressive Power and Inference

Expressive Power: A purely linear model lacks the expressive power of non-linear RNNs. To address this, SSM architectures re-introduce non-linearities in the "vertical" path (i.e., within a single time step), while keeping the "horizontal" recurrence linear. This is typically done by placing an MLP or other non-linear activation functions before or after the SSM layer. This hybrid approach combines the expressive power of non-linearities with the training efficiency of linear recurrences.

Inference: While the parallel convolution view is ideal for training, the recurrent view is highly efficient for inference, especially in auto-regressive tasks.

- To generate the next output, we only need the current state $\vec{h}_t$ and the current input $\vec{x}_{t+1}$.
- We update the state using the simple recurrent formula: $\vec{h}_{t+1} = A\vec{h}_t + B\vec{x}_{t+1}$.
- The entire history of the sequence is compressed into the state vector $\vec{h}_t$. This makes inference very fast (constant time per step, ignoring matrix-vector product costs) and memory-efficient.

This dual formulation gives SSMs the best of both worlds: fast, parallelizable training and efficient, sequential inference. These properties have made them a key component in modern architectures like Mamba, DeltaNet, and large language models such as Qwen.

Lecture 16: Finish State-Space Models

16.1 Recap: State-Space Models (SSMs) for Sequence Modeling

We continue our discussion on State-Space Models (SSMs), which are motivated by two key challenges with Recurrent Neural Networks (RNNs):

1. **Parallelization:** RNNs process sequences sequentially, making it difficult to parallelize training over the time dimension. The computation of the hidden state h_t depends on h_{t-1} .
2. **Long-Range Dependencies:** The sequential application of non-linearities and matrix multiplications can lead to vanishing or exploding gradients, making it hard for RNNs to capture dependencies over long time horizons. The influence of past inputs decays over time.

16.1.1 The SSM Approach: Linear Time-Invariant (LTI) Systems

The core idea behind modern SSMs is to eliminate the "horizontal" non-linearity in the recurrence relation.

- A traditional RNN has the form: $\vec{h}_t = \sigma(A\vec{h}_{t-1} + B\vec{x}_t + \vec{b})$
- By removing the non-linearity σ and bias $\vec{b}$, we obtain a linear system: $\vec{h}_t = A\vec{h}_{t-1} + B\vec{x}_t$

This results in a standard **Linear Time-Invariant (LTI) state-space model**:

$$\vec{h}_t = A\vec{h}_{t-1} + B\vec{x}_t \quad (\text{State equation}) \quad (16)$$

$$\vec{y}_t = C\vec{h}_t + D\vec{x}_t \quad (\text{Output equation}) \quad (17)$$

where A, B, C, D are learned weight matrices that are constant over time.

By unrolling the recurrence, we can express the output $\vec{y}_k$ directly in terms of all previous inputs, without depending on intermediate hidden states:

$$\vec{y}_k = CA^k\vec{h}_0 + \sum_{i=1}^k CA^{k-i}B\vec{x}_i + D\vec{x}_k \quad (18)$$

Assuming an initial state $\vec{h}_0 = \vec{0}$, this becomes:

$$\vec{y}_k = D\vec{x}_k + CB\vec{x}_{k-1} + CAB\vec{x}_{k-2} + \dots + CA^{k-1}B\vec{x}_1 \quad (19)$$

This is a **convolution**. The output at time k is a weighted sum of all inputs up to time k . The convolution kernel (or filter) is defined by the sequence of matrices $(D, CB, CAB, \dots, CA^{k-1}B, \dots)$.

This convolutional form has a major advantage: it can be computed in parallel for all time steps, dramatically speeding up training. A naive convolution over a sequence of length T requires $O(T^2)$ computation. However, we can use the **Fast Fourier Transform (FFT)** to compute the convolution in $O(T \log T)$ time.

16.2 Efficient Parallel Training for SSMs (The S4 Model)

The Structured State-Space Model (S4) builds on this foundation by addressing several practical challenges.

16.2.1 Challenge 1: Computing Powers of Matrix A

Computing the coefficients of the convolution kernel requires calculating powers of the matrix A (A^k). This is computationally expensive and must be done sequentially.

- **Solution:** Add structure to A . The easiest matrix to exponentiate is a **diagonal matrix**.
- We constrain A to be diagonal. This allows us to compute A^k by simply raising each diagonal element to the power of k .
- While this restricts the model to diagonalizable systems, we can learn the basis (eigenvectors) implicitly, allowing the model to represent a large family of linear systems.

16.2.2 Challenge 2: Applying FFT to Vector Sequences

The standard FFT algorithm is designed for scalar sequences, but our inputs $\vec{x}_t$ and outputs $\vec{y}_t$ are vectors. Let d_x , d_y , and d_h be the dimensions of the input, output, and hidden state, respectively. The hidden state dimension d_h is our "memory" capacity.

- **Solution:** Decompose the problem. We can view a sequence of vectors as a stack of scalar sequences.
- For the output vector $\vec{y}_t = [y_t[1], y_t[2], \dots, y_t[d_y]]^T$, we can consider the time evolution of each component $y_t[k]$ separately.
- The k -th component of the output, $y_t[k]$, is a linear combination of all components of all past inputs $\vec{x}_i$ for $i \leq t$.
- This means we can express the sequence of the k -th component, denoted $Seq(y[k])$, as a sum of convolutions involving the sequences of each input component, $Seq(x[l])$:

$$Seq(y[k]) = \sum_{l=1}^{d_x} F_{kl} * Seq(x[l]) \quad (20)$$

where F_{kl} is an appropriate scalar filter and $*$ denotes convolution. Each of these scalar convolutions can be computed efficiently using FFT.

16.2.3 Challenge 3: Reducing the Number of Convolutions

The decomposition above results in $d_x \times d_y$ separate convolutions. If $d_x = d_y = 100$, this means 10,000 convolutions, which is computationally intensive.

- **Solution:** Take inspiration from depthwise/channel-wise convolutions in CNNs. We can prevent channels from mixing within the SSM layer.

- We enforce a structure where each input channel only influences a corresponding set of hidden state dimensions and output channels, without interacting with others.
- Since A is already diagonal, this is achieved by imposing a **block-diagonal structure** on matrices B and C .
- This sets many of the filters F_{kl} to zero, drastically reducing the number of required convolutions and speeding up computation.

16.2.4 Challenge 4: Capturing Long-Range Dependencies

The other major goal of SSMs is to avoid the vanishing history problem of RNNs. This is determined by the eigenvalues of the state matrix A . Since we’ve constrained A to be diagonal, its eigenvalues λ_i are its diagonal entries.

- If $|\lambda_i| > 1$: The corresponding hidden state component will explode exponentially. This leads to numerical instability.
- If $|\lambda_i| < 1$: The state component decays to zero. This is the vanishing history problem we want to avoid.
- If $|\lambda_i| = 1$: The state component’s magnitude is preserved over time. This is the ideal ”sweet spot” for maintaining long-term memory.

If we restrict eigenvalues to be real numbers, the only options are $\lambda = +1$ and $\lambda = -1$. This is a very limited, discrete set, which is not amenable to gradient-based optimization.

- **Solution:** Use **complex numbers** for the eigenvalues. This allows the eigenvalues to lie anywhere on the unit circle in the complex plane, a continuous space perfect for optimization. The complex part introduces rotation, while the magnitude determines scaling.
- To ensure stability (i.e., $|\lambda| \leq 1$), we can parameterize the eigenvalues. For example, by representing them in polar form $re^{i\theta}$ and constraining $r \leq 1$, or by parameterizing the real and imaginary parts. A common parameterization is:

$$\lambda = e^{-\text{ReLU}(p_{\text{real}}) + j \cdot p_{\text{imag}}} \quad (21)$$

Here, p_{real} and p_{imag} are learnable parameters. The $-\text{ReLU}(\cdot)$ term ensures the real part of the exponent is non-positive, which guarantees $|\lambda| \leq 1$.

- **Initialization:** The performance of S4 is very sensitive to the initialization of these eigenvalues. Techniques derived from the HiPPO (High-order Polynomial Projection Operator) framework provide strong initializations that place eigenvalues strategically near the unit circle. In fact, one can achieve good performance by freezing these HiPPO-initialized eigenvalues and only training matrices B and C .

By carefully engineering the system to maintain information over time, S4 models excel at tasks requiring long-range dependencies, outperforming many Transformer-based models on benchmarks like the Long Range Arena.

16.3 Mamba: Selective State-Space Models (S6)

While S4 is powerful, its LTI nature is a limitation. The convolution kernel is fixed and does not depend on the input. This means the way the model looks at the past is static for all tokens. Mamba (also known as S6: Structured Selective State-Space) introduces **selectivity**, allowing the system to adapt its behavior based on the input data.

16.3.1 A New Perspective: Continuous-Time Systems

Mamba's key idea comes from viewing the discrete-time SSM as a discretization of an underlying continuous-time linear system:

$$\dot{\vec{h}}(t) = A_c \vec{h}(t) + B_c \vec{x}(t) \quad (22)$$

To convert this to a discrete-time system, we need to choose a sampling interval, Δ . Using a standard discretization method (like the zero-order hold), we get a discrete-time system $\vec{h}_k = \bar{A} \vec{h}_{k-1} + \bar{B} \vec{x}_{k-1}$ where the new matrices $\bar{A}$ and $\bar{B}$ are functions of the original continuous matrices A_c, B_c and the sampling interval Δ :

$$\bar{A} = e^{A_c \Delta} \quad (23)$$

$$\bar{B} = (e^{A_c \Delta} - I) A_c^{-1} B_c \quad (24)$$

This introduces Δ as a new degree of freedom.

16.3.2 Learning to be Selective with Δ

The sampling interval Δ controls the trade-off between attending to the past state versus the current input. Assume the real parts of the eigenvalues of A_c are negative for stability.

- **Small Δ :**
 - $\bar{A} = e^{A_c \Delta} \approx I$. The new state $\vec{h}_k$ is very close to the old state $\vec{h}_{k-1}$. The system **remembers its history**.
 - $\bar{B} \approx B_c \Delta$. The influence of the current input $\vec{x}_{k-1}$ is small.
- **Large Δ :**
 - $\bar{A} = e^{A_c \Delta} \rightarrow 0$. The old state $\vec{h}_{k-1}$ is effectively forgotten. The system can **reset its memory**.
 - $\bar{B} \rightarrow -A_c^{-1} B_c$. The influence of the current input is large.

The Mamba Idea: Make Δ input-dependent. By learning a function that maps the input $\vec{x}_t$ to a value Δ_t , the model can dynamically decide whether to remember or forget, for each token.

- Since Δ_t must be positive (it's a time interval), a simple linear layer is insufficient.
- We can use an activation function that outputs positive values. A **softplus** function works well:

$$\Delta_t = \text{softplus}(\text{Linear}(\vec{x}_t)) = \log(1 + \exp(\text{Linear}(\vec{x}_t))) \quad (25)$$

Now, the state-space matrices $\bar{A}_t$ and $\bar{B}_t$ change at every time step, making the system **time-varying** and input-dependent. This is the source of selectivity.

16.3.3 Implications for Training

The input-dependent nature of Mamba breaks the LTI property.

- **Loss of FFT:** Since the system is no longer a convolution with a fixed kernel, the $O(T \log T)$ FFT trick cannot be used for training.
- **Parallel Scan:** However, the model retains the key property of having no "horizontal" non-linearity. The recurrence can still be computed in parallel. This is done using a hardware-aware **parallel scan** algorithm. While more computationally complex than FFT, it is still highly parallelizable on modern hardware like GPUs and avoids the sequential bottleneck of RNNs.
- **Inference:** At inference time, the model operates just like an RNN. The state $\vec{h}_t$ can be computed from $\vec{h}_{t-1}$ and $\vec{x}_t$, making it very fast and memory-efficient as it only needs to keep track of the current state, not the entire history.

16.3.4 The Mamba Architecture Block

A typical Mamba block combines the selective SSM with other standard neural network components:

1. The input sequence is passed through a linear projection layer.
2. The result is split into two parallel paths.
3. **Path 1 (SSM Path):** The sequence first goes through a 1D convolution (to capture local context), then a SiLU/Swish activation, and finally into the selective SSM layer. The SSM uses the input to dynamically generate the parameters $\Delta_t, \bar{B}_t, \bar{C}_t$.
4. **Path 2 (Gating Path):** The sequence goes through a linear projection and a SiLU/Swish activation. This path acts as a gate.
5. The outputs of the two paths are multiplied element-wise. This gating mechanism allows the model to control the flow of information.

This architecture effectively merges the long-range context capabilities of SSMs with the local processing power of convolutions and the dynamic information control of gating mechanisms.

Lecture 17: Attention and Transformers

17.1 Introduction: From State-Space Models to Attention

We previously discussed State-Space Models (SSMs) as a way to address challenges with Recurrent Neural Networks (RNNs), particularly the difficulty of parallelizing training. SSMs leverage a convolution perspective, which allows for efficient, parallel training. Their strength lies in the ability to use a very large hidden state, h , to store a vast amount of information about the sequence history. The output matrix, C , can then project this high-dimensional state back to a lower-dimensional output, y , managing the model's parameter count.

Today, we explore another approach to overcoming the limitations of RNNs: **attention mechanisms**. Historically, attention was developed to improve RNN performance before the famous "Attention is All You Need" paper demonstrated that the RNN structure could be discarded entirely. Attention and SSMs represent two distinct evolutionary paths stemming from the challenges of RNNs.

17.2 Sequence-to-Sequence Problems with RNNs

17.2.1 The Encoder-Decoder Architecture

A common application for sequence-to-sequence tasks, such as machine translation, is the **encoder-decoder architecture**.

- **Encoder:** This part of the network processes the input sequence (e.g., an English sentence). It is typically composed of a stack of RNN layers. As it processes the input token by token, it builds up a hidden state that is intended to summarize the entire input sequence.
- **Decoder:** This part of the network generates the output sequence (e.g., the translated Chinese sentence). It is also a stack of RNN layers. It begins with a special "start" token and generates the output sequence one token at a time.

The core idea is to transfer the "meaning" or "context" captured by the encoder to the decoder. In a standard RNN-based architecture, this is done by using the final hidden state of the encoder's layers to initialize the hidden states of the decoder's corresponding layers.

17.2.2 Visualizing the Architecture and the Bottleneck

We can visualize this process by "unrolling" the RNNs through time.

- The horizontal axis represents the time or sequence dimension.
- The vertical axis represents the layers of the network.

The encoder processes the input sequence of length T , $x_1, x_2, \dots, x_T$. After processing the final token x_T , the encoder's final hidden state, h_T , contains a summary of the entire input sentence. This single state vector is then passed to initialize the decoder.

This creates a significant **information bottleneck**. All information about the input sentence—its meaning, grammar, and nuances—must be compressed into a single, fixed-size state vector. For long sentences, this compression is lossy and makes it very difficult for the decoder to generate an accurate translation. The only way to cram more information in is to make the hidden state dimension larger, which makes the network much more difficult and computationally expensive to train. The length of the input sequence processed by the encoder does not have to be the same as the length of the output sequence generated by the decoder.

17.3 Training Sequence-to-Sequence Models

17.3.1 Teacher Forcing

During training, we have access to the ground truth output sequence (the correct translation). Instead of feeding the decoder's own (potentially incorrect) previous output as the input for the next step, we can use the ground truth token. This technique is called **teacher forcing**.

- **Process:** At each time step t of the decoder, we provide the correct token from the training data, y_{t-1} , as input to generate the prediction $\hat{y}_t$.
- **Advantage:** This prevents the model from compounding its own errors during training. If the model generates a wrong word at an early step, feeding that wrong word back in could lead it down a completely incorrect path, making it difficult for gradients to correct the behavior. Teacher forcing keeps the model on track and stabilizes training.

17.3.2 Autoregressive Generation (Inference Time)

At test (or inference) time, we do not have the ground truth translation. The model must generate the output sequence on its own. This is done **autoregressively**.

- **Process:**
 1. The decoder starts with a learned "start" token.
 2. It generates a probability distribution over the vocabulary for the first output token.
 3. A token is chosen from this distribution (e.g., by sampling).
 4. This chosen token is then fed back as the input for the next time step.
 5. This process repeats until the model generates a special "end" token.
- **Sampling:** Instead of greedily picking the token with the highest probability at each step, it is often better to *sample* from the output distribution. This introduces randomness, which can lead to more diverse and often higher-quality outputs. A "temperature" parameter can control the level of randomness: low temperature approaches greedy selection, while high temperature increases randomness.

17.4 Overcoming the Bottleneck: The Motivation for Attention

The information bottleneck is the central problem. How can the decoder access information from the entire input sequence, not just the final hidden state?

17.4.1 Analogy to U-Nets

This problem is analogous to one seen in U-Nets for image segmentation. In a U-Net, an image is progressively downsampled (the "encoder" path) to capture global context, creating a bottleneck at the lowest resolution. It is then upsampled (the "decoder" path) to produce a pixel-level segmentation map. To recover fine-grained spatial information lost during downsampling, U-Nets use **skip connections**, linking layers of the same spatial resolution from the encoder path to the decoder path.

These skip connections work because there is a natural, fixed correspondence: a pixel at position (i, j) in a higher-resolution layer corresponds directly to a feature at position (i, j) in the corresponding lower-resolution layer.

17.4.2 The Challenge in Language

For language translation, we cannot use fixed skip connections. The "appropriate position" to look at in the input sentence changes depending on the word being generated. Word order is not preserved across languages. For example, the third word in a French sentence might correspond to the first or fifth word in its English translation.

The connections cannot be predefined; they must be **dynamic** and depend on the input itself. We need a mechanism that allows the decoder, at each step, to look back at all the encoder's hidden states and decide which ones are most relevant for generating the next word.

17.5 Developing the Attention Mechanism: A Step-by-Step Intuition

We need a learnable, differentiable mechanism that functions like a dynamic lookup table or a "soft" hash table. Let's build up the idea. We'll use the concepts of a **Query**, **Key**, and **Value**.

- **Query (Q)**: Represents the current state of the decoder. It's asking, "What information should I pay attention to in the source sentence?"
- **Keys (K)**: Associated with each token in the source sentence. They are the "indices" we can look up.
- **Values (V)**: Also associated with each token in the source sentence. They contain the actual information content of that token.

17.5.1 Idea -1: Exact Match

Look for an exact match of the Query among all Keys and return the corresponding Value.

- **Problem**: In high-dimensional vector spaces, exact matches are virtually impossible, especially with continuous embeddings and across different languages. At initialization, no matches would be found, yielding no gradient signal to learn from. This approach is a non-starter.

17.5.2 Idea 0: Closest Match

Find the Key that is "closest" (e.g., highest similarity) to the Query and return its corresponding Value.

- **Problem:** This is not differentiable. A small change in the Query vector will likely not change which Key is the closest. This results in piecewise constant behavior, leading to zero gradients almost everywhere, which makes learning via gradient descent impossible.

17.5.3 Idea 1: Weighted Average

Instead of a hard selection of one Key, we can compute a weighted average of all Values. The weights should reflect the similarity between the Query and each Key. This makes the entire operation "soft" and differentiable.

A small change in the Query will now smoothly change its similarity to all Keys, which in turn smoothly changes all the weights and the final output. This provides a rich gradient signal for learning.

- **Similarity Score:** A simple and effective way to measure the similarity between the query vector Q and a key vector K_i is the **dot product**: $Q^T K_i$.
- **Normalization:** The dot products can result in very large numbers. To turn these scores into a well-behaved probability distribution (weights that sum to 1), we apply a **softmax** function across all the similarity scores.

$$\alpha_i = \text{softmax}(Q^T K_i) = \frac{\exp(Q^T K_i)}{\sum_j \exp(Q^T K_j)}$$

- **Final Output:** The output of the attention mechanism is the weighted sum of the Value vectors:

$$\text{Attention}(Q, K, V) = \sum_i \alpha_i V_i$$

17.6 The Attention Formula and its Components

17.6.1 Making it Learnable: Q, K, V Projections

Where do the Query, Key, and Value vectors come from? They are learned linear projections of the input embeddings. For an input vector x , we learn weight matrices W_Q , W_K , and W_V :

$$\begin{aligned} Q &= W_Q x \\ K &= W_K x \\ V &= W_V x \end{aligned}$$

These matrices are the learnable parameters of the attention mechanism, optimized via backpropagation.

17.6.2 Scaled Dot-Product Attention

A key issue with the dot product is that as the dimension of the key vectors, d_k , grows, the magnitude of the dot products can also grow large. This can push the softmax function into regions where it has extremely small gradients, hurting the learning process. To counteract this, we scale the dot products before the softmax:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Here, Q, K, V are matrices containing the queries, keys, and values for all tokens in the sequence. This is the core formula for **Scaled Dot-Product Attention**.

17.6.3 Types of Attention

- **Cross-Attention:** This is the original form used in encoder-decoder models. The **Queries** come from the decoder's state, while the **Keys** and **Values** come from the encoder's output states for all input tokens. This allows every decoder step to attend to every part of the input sentence.
- **Self-Attention:** Here, the Queries, Keys, and Values all come from the **same sequence**. For example, within the encoder, each word can look at all other words in the input sentence to build a more context-rich representation.
- **Masked Self-Attention:** Used in decoders. When generating the i -th word, the model should only be able to attend to words $1, \dots, i$. It cannot "look into the future." This is achieved by masking (setting to $-\infty$) the attention scores for future positions before the softmax is applied.

17.7 Multi-Head Attention

Instead of performing a single attention calculation with high-dimensional Q, K, V vectors, it can be beneficial to perform multiple attention calculations in parallel, each with lower-dimensional vectors. This is called **Multi-Head Attention**.

- **Intuition:** This allows the model to jointly attend to information from different representation subspaces at different positions. A single attention head might learn to focus on syntactic relationships, while another focuses on semantic relationships.
- **Mechanism:**
 1. The original Q, K , and V are each linearly projected into h lower-dimensional representations (one for each "head"). The dimension is typically split, so if the original embedding dimension is d_{model} , each head works with dimension $d_k = d_{model}/h$.
 2. Scaled dot-product attention is applied in parallel for each head, producing h output vectors.
 3. These h output vectors are concatenated and passed through a final linear projection to produce the final output.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Multi-head attention is a standard and powerful component of modern neural network architectures, particularly Transformers, which we will discuss next.

Lecture 18: Transformers

18.1 Recap: Sequence-to-Sequence (Seq2Seq) Models

We begin by recapping the sequence-to-sequence problems, particularly in the context of Recurrent Neural Networks (RNNs).

18.1.1 Encoder-Decoder Architecture

A common architecture for tasks like machine translation is the encoder-decoder model.

- **Encoder:** An RNN processes the input sequence (e.g., a sentence in English) one token at a time, summarizing the information into a final hidden state vector.
- **Decoder:** Another RNN takes the encoder's final hidden state as its initial state. It then generates the output sequence (e.g., the translated sentence) auto-regressively.
 - **Auto-regressive Generation:** At each time step, the decoder produces a probability distribution over the vocabulary. A token is sampled from this distribution and fed as the input to the next time step. This process begins with a special 'start' token.

18.1.2 The Bottleneck Problem

A significant limitation of this architecture is the **bottleneck**. The entire meaning and context of the input sequence must be compressed into a single, fixed-size hidden state vector.

- For long sequences, this compression is lossy and makes it difficult for the decoder to access relevant information from early parts of the input.
- Increasing the size of the hidden state to pass more information makes the model larger and harder to train.

This limitation motivated the development of the attention mechanism, which allows the decoder to look back at the entire input sequence at every step of the generation process.

18.1.3 Training Seq2Seq Models

- **Loss Function:** These models are typically trained using a cross-entropy loss on the predicted probability distributions for the next token.
- **Teacher Forcing:** During training, instead of feeding the model's own (potentially incorrect) prediction to the next time step, we feed the ground-truth token from the target sequence. This stabilizes training and helps the model learn the correct sequence structure more quickly.

18.2 The Attention Mechanism

Attention provides a way to overcome the bottleneck problem by creating direct connections between the decoder and all encoder states.

18.2.1 Conceptual Overview: Queries, Keys, and Values

The attention mechanism is analogous to a differentiable dictionary lookup.

- **Query (Q):** Represents the current piece of information we are focused on (e.g., the current hidden state of the decoder). It's asking a question: "What information from the source is relevant right now?"
- **Keys (K):** A set of vectors associated with the source information (e.g., all encoder hidden states). Each key acts as a label or an identifier for a piece of information.
- **Values (V):** A set of vectors, where each value is paired with a key. Values contain the actual content or information.

The process is as follows:

1. The query is compared with all keys to compute a **similarity score**.
2. These scores are converted into a probability distribution (weights) using a softmax function. These weights determine how much "attention" to pay to each value.
3. The final output is a weighted sum of all the values, based on the computed attention weights.

18.2.2 Calculating Q, K, and V

We don't assume Q, K, and V are given. Instead, we learn them from our input data. Given an input vector $\vec{x}$, we generate Q, K, and V using learned linear transformations:

$$\begin{aligned}\vec{q}_n &= W_q \vec{x}_n + \vec{\beta}_q \\ \vec{k}_m &= W_k \vec{x}_m + \vec{\beta}_k \\ \vec{v}_l &= W_v \vec{x}_l + \vec{\beta}_l\end{aligned}$$

Here, W_q, W_k, W_v are learnable weight matrices, and $\vec{\beta}_q, \vec{\beta}_k, \vec{\beta}_l$ are learnable bias vectors. Gradient descent determines their values during training.

18.2.3 Types of Attention

Cross-Attention: This is the form of attention used in the original encoder-decoder context.

- **Queries** come from the decoder (representing the target sequence state).
- **Keys and Values** come from the encoder (representing the source sequence).
- This allows the decoder, at each step, to look across the entire input sequence to decide what information is most relevant for generating the next output token.

Self-Attention: In self-attention, the queries, keys, and values are all derived from the *same* source sequence.

- This allows each position in a sequence to attend to all other positions in the same sequence.
- It helps the model build richer representations by understanding dependencies and relationships within the sequence itself (e.g., pronoun resolution).

- **Masked (Causal) Self-Attention:** Used in decoders. To prevent the model from "cheating" by looking at future tokens during training, we apply a mask to the attention scores. The mask sets the scores for all subsequent positions to $-\infty$ before the softmax, ensuring a position can only attend to itself and previous positions.

18.3 Scaled Dot-Product Attention

The most common implementation of attention is scaled dot-product attention.

18.3.1 The Attention Formula

Given a query $\vec{q}$ and a set of keys $\vec{k}_j$ and values $\vec{v}_j$, the attention output is calculated as:

1. **Compute scores:** The similarity score is the dot product of the query with each key: $s_j = \langle \vec{q}, \vec{k}_j \rangle$.
2. **Apply Softmax:** The scores are converted to weights using the softmax function: $\alpha_j = \frac{\exp(s_j)}{\sum_i \exp(s_i)}$.
3. **Compute weighted sum:** The output is the weighted sum of the values: $\text{Attention}(\vec{q}, K, V) = \sum_j \alpha_j \vec{v}_j$.

18.3.2 The Need for Scaling

The simple dot product doesn't work well in practice. If the dimension of the key/query vectors, d_k , is large (e.g., 1024), the magnitude of the dot product $\langle \vec{q}, \vec{k}_j \rangle$ can become very large.

- A dot product is a sum of d_k terms. If the components of $\vec{q}$ and $\vec{k}$ have zero mean and unit variance, the variance of their dot product is d_k .
- Large input values to the softmax function push it into regions where the gradient is extremely small (saturation). This makes learning very difficult.

Solution: Scale the dot product scores before the softmax. Inspired by the Central Limit Theorem, we scale by the standard deviation of the sum, which is $\sqrt{d_k}$.

The **Scaled Dot-Product Attention** formula is:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

This scaling prevents the softmax inputs from getting too large, ensuring stable gradients and more effective training.

18.4 Multi-Head Attention

Instead of performing a single attention calculation, it's beneficial to allow the model to jointly attend to information from different representation subspaces. This is the idea behind Multi-Head Attention.

18.4.1 Mechanism

1. The input embedding dimension D is split among H different "attention heads". Each head operates on a smaller dimension of D/H .
2. For each head $i \in \{1, \dots, H\}$, we learn separate weight matrices $(W_{q,i}, W_{k,i}, W_{v,i})$ to project the input into a lower-dimensional query, key, and value space.
3. Each head computes its attention output in parallel:

$$\text{head}_i = \text{Attention}(XW_{q,i}, XW_{k,i}, XW_{v,i})$$

4. The outputs of all heads are concatenated back together.
5. This concatenated result is passed through a final linear projection layer with weight matrix W_o to produce the final output, which has the same dimension as the input.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W_o$$

This allows different heads to specialize and focus on different types of information or relationships within the sequence (e.g., one head for syntactic relationships, another for semantic ones).

18.4.2 Variations for Efficiency

Multi-Query Attention: To save computation and memory, use the same keys and values for all attention heads. Only the query is projected differently for each head.

Grouped-Query Attention: A compromise where heads are divided into groups. Heads within the same group share keys and values.

18.5 The Transformer Architecture

The seminal paper "Attention is All You Need" introduced the Transformer, a model architecture that dispenses with recurrence and convolutions entirely, relying solely on self-attention mechanisms.

18.5.1 Overall Structure

The Transformer follows an encoder-decoder structure, but each component is built from stacked attention-based layers.

18.5.2 The Encoder Block

Each encoder block consists of two main sub-layers:

1. **Multi-Head Self-Attention:** The input sequence attends to itself.
2. **Position-wise Feed-Forward Network (FFN):** A two-layer MLP applied independently to each position.

Figure 1: The Transformer Architecture (from "Attention is All You Need")

Left side (Encoder): A stack of N identical layers. Each layer has a multi-head self-attention module followed by a feed-forward network.

Right side (Decoder): A stack of N identical layers. Each layer has a masked multi-head self-attention module, followed by a multi-head cross-attention module (attending to the encoder's output), followed by a feed-forward network.

Connections: Residual connections are used around each sub-layer, followed by layer normalization. The final output of the encoder stack is used as the key and value for the cross-attention module in *every* decoder layer.

A residual connection is applied around each of the two sub-layers, followed by layer normalization. The output of a sub-layer is $\text{LayerNorm}(x + \text{Sublayer}(x))$.

Note: More recent architectures often use Pre-LN, where layer normalization is applied *before* the sub-layer: $x + \text{Sublayer}(\text{LayerNorm}(x))$. This can lead to more stable training.

18.5.3 The Decoder Block

Each decoder block has three sub-layers:

1. **Masked Multi-Head Self-Attention:** The decoder attends to the output sequence it has generated so far. Masking ensures it cannot see future tokens.
2. **Multi-Head Cross-Attention:** The queries come from the previous sub-layer, but the keys and values come from the output of the **final encoder block**. This is where the decoder incorporates information from the source sentence.
3. **Position-wise FFN:** Same as in the encoder.

Similar to the encoder, residual connections and layer normalization are used around each sub-layer.

18.6 Embedding and Positional Encoding

18.6.1 Input/Output Embeddings

- **Input:** The model receives input tokens as one-hot encoded vectors of vocabulary size (e.g., 32,000).
- **Input Embedding:** A learnable linear layer projects these high-dimensional one-hot vectors into a lower-dimensional, dense embedding space (e.g., dimension $d_{\text{model}} = 1024$). Since the input is one-hot, this operation is equivalent to a lookup in an embedding matrix. For good initialization, the columns of this matrix should have an RMS norm of approximately 1.
- **Output De-embedding:** After the final decoder block, another linear layer projects the model's output representation back to the vocabulary size to produce logits for each possible next token. A softmax is then applied to get probabilities.
 - **Initialization:** The input to this layer is normalized. To prevent the output logits from being too large (which would saturate the softmax), the rows of this "de-embedding" matrix

should be initialized with small values. A standard approach is to initialize entries with a standard deviation of $1/\sqrt{d_{\text{model}}}$.

18.6.2 Positional Encoding

The self-attention mechanism is permutation-invariant; it does not have any inherent sense of token order or position. Since word order is crucial in language, we must explicitly inject this information into the model.

Original Sinusoidal Encoding (Obsolete): The original Transformer paper added fixed vectors to the input embeddings. These vectors were generated using sine and cosine functions of different frequencies, uniquely encoding each position.

No Positional Encoding (NOPE): It was discovered that for many tasks, models without explicit positional encoding perform surprisingly well. It's hypothesized that the model can infer relative positions through other means (e.g., the causal mask in the decoder implies order).

Learned Relative Positional Encoding: This approach modifies the attention score calculation directly. A learnable bias term b_{t-i} is added to the attention logit for a query at position t and a key at position i . The bias depends only on the relative distance $t - i$, and these biases are learned during training.

$$\text{score}(q_t, k_i) = \frac{\vec{q}_t^T \vec{k}_i}{\sqrt{d_k}} + b_{t-i}$$

Rotary Positional Encoding (RoPE): The current state-of-the-art method. Instead of adding to the scores, RoPE modifies the queries and keys directly using rotation matrices.

- The embedding vector at each position is treated as a sequence of 2D vectors (or complex numbers).
- Each 2D component of the query and key vectors is rotated by an angle that is a function of its absolute position.
- A key property of rotations is that the inner product between two vectors rotated by angles θ_m and θ_n depends only on the relative angle difference, $\theta_m - \theta_n$.
- This elegantly injects relative position information into the attention score $\langle R_t \vec{q}, R_i \vec{k} \rangle$ without changing the norms of the vectors. This has been shown to be very effective.

Lecture 19: Finish Transformers

19.1 Recap and Introduction

This lecture aims to deconstruct modern deep learning architectures, such as the Qwen model, so that every component is understood. We will build upon our previous discussion of positional encoding and explore various architectural innovations that have led to the current state-of-the-art in large language models.

19.2 Rotary Positional Encoding (RoPE)

Attention mechanisms are permutation-invariant, meaning they do not inherently understand the order of tokens in a sequence. Positional encodings are introduced to solve this. We previously discussed several methods:

- **Absolute Positional Encoding:** Adding a unique vector to each token embedding based on its absolute position. This was found to be suboptimal.
- **NoPE (No Positional Encoding):** Simply not adding any positional information. This surprisingly outperformed absolute encodings in some cases.
- **Learned Relative Positional Encoding:** Modifying the attention score directly with a learned bias that depends on the relative distance between the query and key tokens. This was an improvement but has been superseded.

The current state-of-the-art is **RoPE (Rotary Positional Encoding)**.

19.2.1 The Core Idea of RoPE

The key idea behind RoPE is to modify the query vector $\mathbf{q}$ and key vector $\mathbf{k}$ based on their absolute positions, such that their inner product (which determines the attention score) depends only on their *relative* position.

The attention score is computed via an inner product, $\langle \mathbf{q}, \mathbf{k} \rangle$. We know that for two vectors, this can be expressed as:

$$\langle \mathbf{q}, \mathbf{k} \rangle = \|\mathbf{q}\|_2 \|\mathbf{k}\|_2 \cos(\theta)$$

where θ is the angle between them. RoPE leverages this by using positional information to *rotate* the query and key vectors. If we can modulate the angle between them based on their positions, we can encode positional information into the attention score.

19.2.2 RoPE in Two Dimensions

Consider a 2D case where $\mathbf{q}$ and $\mathbf{k}$ are two-dimensional vectors. A rotation by an angle ϕ can be achieved by multiplying with a rotation matrix:

$$\mathbf{R}_\phi = \begin{bmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{bmatrix}$$

Let the query be at position t and the key at position i . We apply rotations to them based on their positions:

- Query at position t is transformed to $\mathbf{q}'_t = \mathbf{R}_{\omega t} \mathbf{q}_t$.
- Key at position i is transformed to $\mathbf{k}'_i = \mathbf{R}_{\omega i} \mathbf{k}_i$.

The new inner product becomes:

$$\langle \mathbf{q}'_t, \mathbf{k}'_i \rangle = \langle \mathbf{R}_{\omega t} \mathbf{q}_t, \mathbf{R}_{\omega i} \mathbf{k}_i \rangle = (\mathbf{R}_{\omega t} \mathbf{q}_t)^T (\mathbf{R}_{\omega i} \mathbf{k}_i) = \mathbf{q}_t^T \mathbf{R}_{\omega t}^T \mathbf{R}_{\omega i} \mathbf{k}_i$$

Since rotation matrices are orthogonal, $\mathbf{R}_{\omega t}^T = \mathbf{R}_{-\omega t}$. The product of two rotation matrices is a rotation by the sum of their angles:

$$\mathbf{R}_{-\omega t} \mathbf{R}_{\omega i} = \mathbf{R}_{\omega(i-t)}$$

Thus, the inner product becomes:

$$\langle \mathbf{q}'_t, \mathbf{k}'_i \rangle = \mathbf{q}_t^T \mathbf{R}_{\omega(i-t)} \mathbf{k}_i$$

This shows that the final score depends only on the relative position, $i - t$.

19.2.3 Extending RoPE to Higher Dimensions

Transformer embedding dimensions are much larger than 2 (e.g., 1024, 4096). To apply this rotation idea to a d -dimensional vector, RoPE groups the vector's dimensions into pairs.

- A d -dimensional vector is treated as a concatenation of $d/2$ two-dimensional vectors (or complex numbers).
- A different rotation is applied to each 2D block.

This is implemented using a block diagonal matrix $\mathbf{M}_t$, where each block $\mathbf{M}_j$ on the diagonal is a 2D rotation matrix:

$$\mathbf{M}_t = \begin{bmatrix} \mathbf{M}_1 & & & \\ & \mathbf{M}_2 & & \\ & & \ddots & \\ & & & \mathbf{M}_{d/2} \end{bmatrix} \quad \text{where} \quad \mathbf{M}_j = \begin{bmatrix} \cos(\theta_j t) & -\sin(\theta_j t) \\ \sin(\theta_j t) & \cos(\theta_j t) \end{bmatrix}$$

The rotation frequencies θ_j are pre-selected and fixed; they are not learned parameters. They are chosen across a range of scales (often logarithmically spaced).

- **Low Frequencies:** For small θ_j , the angle $\theta_j t$ changes slowly with position t . These channels are less sensitive to distance and can capture long-range dependencies.
- **High Frequencies:** For large θ_j , the angle changes rapidly. These channels are very sensitive to small changes in position and capture local, short-range information.

The model learns which information to place in which frequency channels by learning the query and key weights, effectively specializing different parts of the embedding space for different positional scales.

19.3 Key Transformer Architectures

19.3.1 GPT-3: The Decoder-Only Model

The Generative Pre-trained Transformer (GPT) family represents decoder-only models, designed primarily for text generation.

- **Architecture:** It consists of a stack of transformer decoder blocks. Unlike the original transformer, there is no encoder and no cross-attention. Each block contains:
 1. **Masked (Causal) Self-Attention:** This is crucial. To predict the next token, a token at position i can only attend to tokens at positions $j \leq i$. This prevents the model from "cheating" by looking at future tokens during training.
 2. **Feed-Forward Network (FFN):** A standard multi-layer perceptron (MLP) that processes each position independently.
 3. **Normalization and Residual Connections:** Used throughout the block for stable training.
- **Training:** GPT is trained on a next-token prediction task in an auto-regressive manner.
 - The input is a sequence of tokens, and the target is the same sequence shifted one position to the left.
 - For an input token at position i , the model is trained to predict the token at position $i + 1$.
 - The loss function is typically cross-entropy, calculated at every position in the sequence. This makes training very data-efficient, as every token contributes to the loss.
- **Inference (Generation):**
 - Generation is sequential. The model is given a starting prompt.
 - It predicts the next token, which is then appended to the input sequence.
 - This new, longer sequence is fed back into the model to generate the subsequent token, and so on.
 - **KV-Caching:** During this process, the computed keys and values for past tokens can be cached and reused, making inference much more efficient.
 - **Sampling Strategies:** To select a token from the output probability distribution, common methods include:
 - * **Top-k Sampling:** Sample only from the k most probable tokens to avoid generating very rare or nonsensical words.

- * **Beam Search:** Instead of greedily picking the best next token, maintain a "beam" of k most probable sequences at each step. This allows the model to explore different generation paths and potentially find a globally more probable sequence.
- **Scale of GPT-3 (Largest Version):**
 - **Parameters:** ~175 billion
 - **Layers:** 96
 - **Attention Heads:** 96
 - **Embedding Dimension:** 12,288
 - **Context Length:** 2048 tokens

19.3.2 BERT: The Encoder-Only Model

BERT (Bidirectional Encoder Representations from Transformers) is an encoder-only model designed to learn deep bidirectional representations of language. It is not for generation but for creating powerful embeddings for downstream tasks.

- **Architecture:** A stack of transformer encoder blocks. Crucially, it uses **full (non-causal) self-attention**, allowing every token to attend to every other token in the sequence (past and future).
- **Pre-training Tasks:** BERT is pre-trained on two self-supervised tasks:
 1. **Masked Language Modeling (MLM):**
 - About 15% of the input tokens are randomly replaced with a special '[MASK]' token.
 - The model's objective is to predict the original identity of these masked tokens based on the surrounding unmasked context. This forces the model to learn a deep understanding of linguistic context.
 2. **Next Sentence Prediction (NSP):**
 - The model is given two sentences, A and B, separated by a '[SEP]' token.
 - The task is to predict whether sentence B is the actual sentence that follows A in the original text, or if it is a random sentence from the corpus.
 - This was intended to help the model understand sentence relationships, but later research found it to be less impactful, and many modern encoder models have dropped it.
- **Fine-tuning:** After pre-training, BERT can be fine-tuned for various downstream tasks. This typically involves adding a small, task-specific output layer on top of the pre-trained BERT encoder.
 - **Sentence Classification (e.g., Sentiment Analysis):** The final hidden state corresponding to the special '[CLS]' token (prepended to every input) is fed into a classifier. The model is fine-tuned to predict a label for the entire sentence (e.g., positive/negative review).
 - **Token Classification (e.g., Named Entity Recognition):** The final hidden state for each token is fed into a classifier to predict a label for that specific token (e.g., person, location, organization).

19.4 Modern Architectural Components

19.4.1 Gated Linear Units (GLU)

The standard Feed-Forward Network (FFN) in a transformer block is often replaced by a Gated Linear Unit (GLU) or its variants.

- **Mechanism:** The input $\mathbf{x}$ is passed through two parallel linear layers. The output of one layer is passed through a gating activation function (like sigmoid), and the result is element-wise multiplied with the output of the other linear layer.

$$\text{GLU}(\mathbf{x}) = (\mathbf{x}\mathbf{W} + \mathbf{b}) \odot \sigma(\mathbf{x}\mathbf{V} + \mathbf{c})$$

Here, $\odot$ denotes element-wise multiplication, and σ is the sigmoid function. The second term acts as a "gate," controlling which information from the first term passes through.

- **SwiGLU:** A popular and effective variant that uses the Swish activation function as the gate. Swish is defined as:

$$\text{Swish}_\beta(x) = x \cdot \sigma(\beta x)$$

where β can be a learned parameter. The SwiGLU operation is:

$$\text{SwiGLU}(\mathbf{x}) = (\mathbf{x}\mathbf{W} + \mathbf{b}) \odot \text{Swish}(\mathbf{x}\mathbf{V} + \mathbf{c})$$

- **Intuition:** GLUs have been empirically shown to improve performance. While a full theoretical explanation is elusive (leading to the famous "divine benevolence" quote from a paper by Shazeer), one intuition is that they can more easily model multiplicative interactions and approximate piecewise quadratic functions, offering more expressive power than the piecewise linear ReLU.

19.4.2 Mixture of Experts (MoE)

Mixture of Experts (MoE) is a technique to increase model capacity without a proportional increase in computational cost for each forward pass.

- **Motivation:** Standard transformer models are "dense," meaning all parameters are used for processing every token. This can be computationally expensive.
- **Core Idea:** Replace the single large FFN in a transformer block with a set of smaller FFNs, called "experts." For each token, a lightweight "router" network decides which expert(s) should process it.

$$\text{Token} \xrightarrow{\text{Router}} \text{Selects Top-K Experts} \rightarrow [\text{Expert}_1, \text{Expert}_2, \dots] \xrightarrow{\text{Combine}} \text{Output}$$

- **Mechanism:**
 1. The router is typically a small linear layer that outputs a probability distribution over the available experts for a given token.
 2. In modern MoE models, the router selects the top-K experts (e.g., K=2 or 4) with the

highest probabilities.

3. The token is passed through each of these K experts.
 4. The final output is a weighted sum of the outputs from the selected experts, where the weights are the normalized probabilities from the router.
- **Advantages:** This allows for a massive increase in the total number of parameters in the model, while keeping the number of active parameters (and thus the computational cost) for any single token constant. This leads to the concept of "sparse activation."
 - **Challenges:** Training MoE models can be complex due to challenges like load balancing (ensuring all experts receive a similar amount of work) and training stability.

19.5 Putting It All Together: A Modern Architecture

Modern LLM architectures like Qwen, Llama 3, and Gemini are not monolithic but are compositions of the advanced components discussed. A typical modern block might feature:

- **Pre-Normalization:** Normalization layers (e.g., RMSNorm) are applied *before* the attention and FFN sub-layers, which has been found to improve training stability.
- **Rotary Positional Encoding (RoPE):** This is the standard for positional information. Some models use variants like *Partial RoPE*, where RoPE is only applied to a fraction of the embedding dimensions.
- **Gated Linear Units (GLU):** SwiGLU is commonly used in place of the standard ReLU-based FFN.
- **Mixture of Experts (MoE):** Increasingly, dense FFN layers are being replaced with MoE layers to scale up model size efficiently.
- **Hybrid Structures:** Some architectures, like Qwen, are hybrids, mixing standard attention layers with other sequence-modeling mechanisms like state-space models (e.g., Mamba, Gated Delta Net) within the same network.

By understanding these individual components, we can deconstruct and comprehend the design principles behind the latest and most powerful large language models. The field continues to evolve, with new combinations and variations of these fundamental building blocks constantly being explored.